



中國石油大學 (华东)
CHINA UNIVERSITY OF PETROLEUM

本科毕业设计（论文）

题 目：海洋环境要素场可视化方法研究

姓 名：覃艺明

学 号：2109060313

学 院：海洋与空间信息学院

专 业：地理信息科学（实验）

指导教师：李连伟

2025 年 6 月 10 日

学位论文原创性声明

本人所提交的学位论文《海洋环境要素场可视化方法研究》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

论文作者（签名）：覃艺明

2025年6月10日

指导教师确认（签名）：李连伟

2025年6月10日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

论文作者（签名）：覃艺明

2025年6月10日

指导教师（签名）：李连伟

2025年6月10日

海洋环境要素场可视化方法研究

摘 要

海洋环境要素场数据（温度、海面高度、流场等）具有多维度、多时空尺度特性，其高效可视化对海洋资源开发、环境监测及灾害预警等决策具有重大战略意义。然而，传统单机可视化工具难以支撑 Web 环境下千万级网格数据的实时渲染，且现有 Web 方案在三维动态表达、交互效率方面存在显著瓶颈。为此，本文设计基于 Cesium 框架的海洋环境要素场 Web 端可视化创新方法体系，突破海量数据动态渲染的技术壁垒。

本文通过构建多维数据映射模型与轻量化渲染架构，实现了海洋标量场/矢量场的高效可视化。本文的主要工作如下：

（1）NetCDF 数据处理。开发网格数据二维化算法，将原始一维数组按经纬度索引重构为空间坐标-强度值映射矩阵；完成数据的采样、过滤处理并以 JSON（JavaScript Object Notation，JavaScript 对象表示法）的形式作为数据服务发布。

（2）可视化方法研究。针对标量场数据，设计离散数据实例化颜色映射法（支持点/柱/多面体渲染）、纹理映射法（集成最邻近/双线性/双三次插值算法）及海面波浪生成法（融合光线追踪与二分法高度场计算）。针对矢量场数据，开发点图标法实现流畅静态模拟，开发粒子系统法实现流场动态模拟。

（3）高性能、高兼容性系统构建。基于 React、Cesium 框架构建 B/S（Browser/Server，浏览器/服务器模式）可视化系统，支持 NetCDF 文件数据文件结构的查看以及上传进度条显示，实现任何尺寸下的响应式布局、国际化语言处理、主题切换等功能。实验环境下，支持千万级网格数据场景下仍可保持流畅渲染。

关键词：Cesium 框架；Web 可视化；可视化方法；标量场；矢量场

The design and implementation of the linear form

Abstract

Marine environmental field data (e.g., temperature, sea surface height, current fields) exhibit multi-dimensional and multi-spatiotemporal scale characteristics. Their efficient visualization holds significant strategic importance for decision-making in marine resource development, environmental monitoring, and disaster early warning. However, traditional standalone visualization tools struggle to support real-time rendering of tens of millions of grid-level data in web environments, while existing web solutions face notable bottlenecks in 3D dynamic expression and interaction efficiency. To address these challenges, this thesis proposes an innovative methodology for web-based visualization of marine environmental fields based on the Cesium framework, overcoming technical barriers in massive data dynamic rendering.

By constructing a multi-dimensional data mapping model and a lightweight rendering architecture, this study achieves efficient visualization of marine scalar/vector fields. The main contributions are as follows:

(1) NetCDF data processing: Developed a grid data two-dimensionalization algorithm to reconstruct raw 1D arrays into spatial coordinate-intensity mapping matrices through latitude-longitude indexing; implemented data sampling and filtering with results published as JSON-based data services.

(2) Visualization methodology: For scalar fields, proposed discrete data instantiation color mapping (supporting point/column/polyhedron rendering), texture mapping (integrating nearest-neighbor/bilinear/bicubic interpolation), and sea surface wave generation (combining ray tracing with binary search height-field computation); for vector fields, developed point-icon methods for static simulation and particle system techniques for dynamic flow field simulation.

(3) High-performance compatible system: Constructed a B/S visualization system using React and Cesium frameworks featuring NetCDF file structure inspection, upload progress

display, responsive layout at any display size, internationalization support, and theme switching. Experimental results demonstrate smooth rendering exceeding 40 frames per second with tens of millions of grid data points.

Keywords: Cesium framework; Web visualization; Scalar field visualization; Vector field visualization

目 录

第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.2.1 海洋环境要素标量场数据可视化方法	2
1.2.2 海洋环境要素矢量场数据可视化方法	3
1.2.3 基于 Web 的地理数据可视化	4
1.3 研究内容和技术路线	6
1.3.1 研究内容	6
1.3.2 技术路线	6
1.4 本文组织结构	8
第 2 章 海洋环境要素场可视方法	9
2.1 海洋环境要素标量场可视化方法	9
2.1.1 离散数据实例化颜色映射法	9
2.1.2 纹理映射法	10
2.1.3 海面波浪生成法	14
2.2 海洋环境要素矢量场可视化方法	18
2.2.1 点图标法	18
2.2.2 粒子系统法	19
第 3 章 海洋环境要素场可视化系统实现	20
3.1 系统实验环境	20
3.2 可视化系统架构设计	20
3.2.1 系统总体架构	20
3.2.2 前端系统设计	21
3.2.3 后端服务设计	27
3.3 海洋环境要素场数据处理	29
3.3.1 NetCDF 格式数据	29

3.3.2 数据服务化..... 30

3.4 海洋环境要素标量场可视化实现..... 32

3.4.1 离散数据实例化颜色映射法..... 32

3.4.2 纹理映射法..... 34

3.4.3 海面波浪生成法..... 39

3.5 海洋环境要素矢量可视化实现..... 40

3.5.1 点图标法..... 40

3.5.2 粒子系统法..... 41

第 4 章 总结与展望..... 44

4.1 总结..... 44

4.2 展望..... 45

致 谢..... 46

参考文献..... 47

第 1 章 绪论

1.1 研究背景及意义

海洋环境是极其复杂的环境，存在多种海洋环境要素场数据，其中标量场数据有海表温度、海表盐度、海表叶绿素浓度等，矢量场数据有海洋流场、海洋风场等。数值建模方法与计算能力的提升促使海洋环境要素场数据呈现爆发式增长，在数据存储方面比较有代表性的是 NetCDF（Network Common Data Format，网络通用数据格式），然而这些数据特征是多时间、多变量、多维并且数据体量大，人们无法直观从这些数据中得到海洋要素场的关键信息。视觉是人们获取外部世界的重要方式，80%以上的外来信息都是经过视觉加工的^[1]，因此海洋环境要素场数据可视化研究变得逐渐热门。

WebGIS 技术的高速发展，特别是 Cesium 三维 GIS 开源地理框架的出现，使得三维可视化成为一个重点研究方向。对海洋环境要素场数据可视化理论的研究经历了二维、三维到多维再到面向知识发现的可视化的历程。从应用层面看，单机及 C/S（Client/Server architecture，客户端/服务器模式）架构的海洋环境要素场数据可视化工具对于多维的海洋要素场数据动态可视化基本实现，但此架构开发的系统具有维护更新难、用户环境配置困难等问题。基于 Web 的可视化技术可以完美的解决这些问题，在 Web 端开发者只需更新服务器端，用户即可自动获取最新功能，无需安装额外软件，也无需电脑拥有很高的配置，因为复杂的数据算法计算是在云端服务器上运行的，仅需浏览器即可查看可视化效果。

本研究将基于 WebGL（Web Graphics Library，Web 图形库）渲染管线系统，结合海洋环境要素场可视化方法理论，利用 Cesium、React、Nodejs 开源框架设计并实现海洋环境要素场数据可视化平台，目的是构建一个基于 NetCDF 文件的高效率的海量海洋要素场数据可视化平台，实现包括海洋环境要素标量场数据和矢量场数据的可视化，本研究将为 Web 端海洋数据可视化体系构建提供理论支撑与技术范式。

1.2 国内外研究现状

1.2.1 海洋环境要素标量场数据可视化方法

海洋环境要素标量数据在海洋学领域已有广泛应用。Web 端海洋环境要素标量场数据可视化的研究主要集中在多维标量场数据的压缩传输及三维标量场 Web 端可视化两个方面。

对于 Web 环境数据的压缩传输,网络环境中通常使用的数据压缩方法有 JPEG 编码(有损)、PNG 编码(无损)、视频编码、GZIP 压缩和 JSON (JavaScript Object Notation, JavaScript 对象表示法)数据。汪小琪^[1]通过三种网络环境实验证明,使用 YUV 4:4:4 色度子采样模型 VP9 编码器的视频压缩方法传输效率最好,在网络环境较好的情况下,PNG 编码压缩的效果也不错,GZIP 压缩的传输效率最差。

对于三维标量场 Web 端可视化,常用的方法有颜色映射法、等值线法等^[2]。颜色映射法通过数值-色彩转换增强数据可读性,以帮助观察者更直观地理解复杂数据的分布和趋势,常见的颜色映射方式包括:线性映射、分级映射、渐变映射。对于屏幕上的像素而言,其颜色值是根据像素位于网格位置的顶点索引值插值计算得到,输出的二维图像的可视化效果取决于颜色的传输函数设置,颜色的变化最终反映的是研究对象的分布特征^[3]。陈超^[4]根据海水温度盐度密度模型,采用 OpenGL 绘制引擎,基于剖面对模型中每一个三角面片的顶点着色,实现了海水温度盐度密度的颜色映射;Pragensia^[5]在 Web 程序中采用颜色映射的方法,并对研究对象之间的关联进行详细的描述;乔洁雯^[6]提出了一种自适应颜色映射方法,并将其应用在数据可视化中;张广宇^[8]通过社会实验(问卷)标定人眼感知边界,将 CIEDE2000 色差公式^[7]转化为工程可用的阈值(0.1),完成了海洋标量数据的颜色映射可视化。等值线法利用标量场数据中相等或相近数值的点,按照一定顺序生成平滑的曲线。通过定义区间间隔,控制绘制不同曲线的密度。气象海洋中常见的标量数据包括等温线、等压线等,这些线要素展示了气候变化特征,以及海底特定区域内环境分布和变化规律。黄国满^[9]将不相关的运算分配到不同的处理器中,实现了等值线的快速生成;刘丁^[10]基于 OpenGL 顶点着色的颜色编码技术,通过对 GPU 渲染管线进行编程大大增加可视化的逼真度、提高模拟的效率;陈超^[4]在规则网格的基础上,通过绘制等值线,对海水温度、盐度等标量数据进行可视化;王想红^[12]在 NetCDF

数据格式的基础上,通过等值面的方法在三维虚拟地球上进行标量数据可视化的表达与分析;宋涛等^[13]基于动态域等值线的方法实现标量场多目标跟踪控制;郑红波等^[14]提出了基于邻接关系的等值线快速追踪算法。

1.2.2 海洋环境要素矢量场数据可视化方法

矢量数据的可视化在气象学、航空航天、生物医学中有广泛应用^{[15][16]}。矢量数据可视化需同时表达方向与强度信息,主流方法一般包括点图标法、矢量线法、纹理映射法、粒子动画方法。

点图标法核心思想是针对数据场中的每个采样点,根据矢量场的大小和方向属性,分别使用图标的大小(或颜色)和方向进行对应。因此,所选用的图标必须具备能够表达大小和方向的特性。该方法适用于数据分布均匀、连续性好的情况,比较容易实现,可视化效果清晰,但当数据不均匀时,需要标记的点数会大大增加,从而增加渲染的负担,并且也容易让各箭头相互遮挡,可视化效果差。郭晓非等^[17]在 Cesium 中利用 Primitive API 绘制箭头图标,根据风速大小映射颜色,并通过克里金插值填补低分辨率数据空缺,但数据量增大时箭头重叠严重;国外研究中,VentuSky 平台采用自适应点图标密度策略,通过地图缩放层级动态调整箭头数量,但存在符号方向与复杂流场匹配精度不足的问题。

矢量线法是一种用线型几何来描述矢量场的方法,这些几何线条包括流线、迹线等。通过从矢量场中提取线数据,并对其进行处理(例如平滑处理),然后进行绘制,以反映各种场线的物理意义。几何形状的可视化方法主要采用矢量线对矢量场进行表达,矢量线表征流场具有精确、连续性好等特点,但种子点的选取对可视化效果影响很大^[18],矢量线数量的增加会导致计算量迅速增加^[19],故该方法无法适用于 Web 端大规模矢量场的可视化。

基于纹理平流的可视化方法是一种通过将纹理中的像素或网格顶点沿矢量场方向移动来表征矢量场各种运动特性的方法。在利用纹理法绘制海流可视化时,尽管纹理图像可以反映海流场的整体情况,但有时在表示方向时存在二义性。此外,纹理法的计算量大,计算方法复杂,通常需要特定的加速算法或利用图形硬件进行加速,因此对计算机硬件性能要求较高^[20]。海洋环境要素长的数据体量大、并且 Web 环境下受限于其沙箱机制无法实现本地系统级硬件控制,所以该方法不太适用于 Web 端大规模矢量场的

可视化。

Reeves^[21]提出粒子系统的概念，由粒子的运动来反映现实的矢量场流向，研究者们一次为基础，将粒子系统广泛用于各种物理现象的模拟当中，例如爆炸、烟花、气体泄漏^{[22][23][24]}，相比于传统的点图标映射法、矢量线方法，该方法是动态的，动画效果较好，能够比较直观地感受到物体的运动趋势，但该方法对空间分辨率高的数据渲染动画帧数低^[25]；Jobard 等^[26]提出流线均匀化算法，通过控制种子点密度避免视觉重叠，但未考虑海洋岸线边界约束；张雅静^[27]针对三角网格数据提出基于信息熵的种子点优化方法，根据网格面积动态调整粒子密度，解决了近岸区域流线过密问题，但算法复杂度较高；张永强^[28]提出基于 WebGL 的四阶龙格库塔算法改进粒子轨迹生成，通过调整积分步长提升路径平滑度，并结合 WebWorker 多线程加速计算，但其全球尺度风场渲染帧率仍受限于数据分块调度效率；相较于传统的可视化方法生成静态图像，粒子系统法可视化结果以动画方式表达，不关注整体的视觉表现而以粒子的运动对矢量场进行表达，算法平均时间复杂度低，更适合 Web 环境下对矢量场的模拟。

1.2.3 基于 Web 的地理数据可视化

传统 C/S 架构的海洋可视化系统（如 Google Ocean）依赖本地计算资源，难以支持多用户并发访问。B/S（Browser/Server，浏览器/服务器模式）架构逐渐成为主流，Cesium 与 ArcGIS API for JavaScript 是代表性工具。Web 可视化技术因跨平台、易共享等优势成为研究热点，其核心挑战在于数据组织、渲染效率与交互设计。

Web 端可视化底层是通过 GPU（Graphic Processing Unit，图形处理器）来完成的，现在的显卡需要安装显卡驱动程序，通过显卡驱动程序暴露的 API（Application Programming Interface，应用程序编程接口），就可以操作 GPU 完成图形处理器的操作，显卡驱动和普通编程界的汇编语言一样，比较底层不好写，于是 OpenGL 腾空出世，负责上层接口的封装与下层显卡驱动打交道，其 2006 年交由 Khronos 组织进行管理。2011 年，WebGL 1.0 发布（基于 OpenGL ES 2.0 标准），其使用 JavaScript 语言绑定调用 OpenGL ES 中的 API，通过前端 HTML5（HyperText Markup Language 5，超文本标记语言）中的 canvas 标签元素实现功能，使得 Web 开发人员能够不使用任何擦插件的前提下，创建 3D 页面应用，WebGL 自发布起就受到热烈追捧，著名的 3D 可视化框架 Cesium.js、Three.js、Babylon.js 就是基于此搭建发布的。在 Khronos 的支持下，WebGL 不断发展于

2017 年发布了 2.0 版本。技术是不断发展的, 由于 OpenGL 发布于计算机图形学的早期, 全局状态机类似的设计已经与现代 GPU 架构脱节, 无法发挥 GPU 最优的能力, 而 WebGL 的设计来源于 OpenGL 子集标准, 因此也同样面临类似的问题, 所以新的图形 API WebGPU 也呼吁而出。WebGL 的总管家对象是 Canvas 元素的 gl 变量, 而 Canvas 元素只能在 JavaScript 上下文中获取, JavaScript 是运行在浏览器的渲染主线程的, 其执行过程会阻碍浏览器的渲染, 这就导致只能在主线程调度 GPU, 浏览器提供的 WebWorker 多线程技术只能用来处理数据, WebGPU 改变了总管家对象的获取方式, 其总管家对象在 WebWorker 中也可以访问, 实现了 CPU 端多线程调度 GPU 的能力, 渲染效率大大提升。目前 Cesium 的最新版本 1.129.0 是基于 WebGL2.0 来实现的, 并没有应用并支持 WebGPU, 地理数据比较庞大, 性能开销大, 所以未来基于 Web 数据的可视化应是基于 WebGPU 实现的, 然而路途还很遥远, 截至 2022 年, 各大主流浏览器厂商才支持 WebGL2.0。

国外研究中, Cameron Beccari 团队开发的 Earth 可视化平台 (<https://earth.nullschool.net>) 率先实现基于 WebGL 的全球风温场实时渲染, 数据通过 JSON 格式动态加载, 但其标量场仅支持二维平面展示, 缺乏三维深度信息。

Cesium 与 Mapbox GL JS 是主流 Web 三维可视化框架。孙强等^[29]基于 Cesium 实现海洋标量场时空动态可视化, 通过瓦片化数据组织与 LOD (Level of Detail, 细节层次) 调度优化渲染性能, 但未支持垂向分层数据。2023 年, Cesium 团队推出 3D Tiles Next 标准^[30], 支持点云与体数据高效传输, 为海洋多维数据可视化提供新思路。WebGL 与 WebGPU 是底层图形接口, 后者通过异步计算提升并行效率。

大屏可视化需兼顾全局概览与细节分析, 视域联动与多视图协同是关键技术。NASA (National Aeronautics and Space Administration, 美国国家航空航天局) 的 GIBS 系统支持多视口同步展示不同时空尺度数据, 但交互响应速度较慢。2022 年, 张广宇^[8]提出基于人眼色差识别的时变阈值筛选方法, 通过减少渲染数据量提升大屏流畅度, 但自适应屏幕分辨率的动态调度算法仍需优化。

综上, 当前海洋环境要素场可视化研究正从传统单机与 C/S 架构向轻量化、跨平台的 B/S 架构加速转型, 依托 WebGL、Cesium 等开源框架实现无插件三维渲染成为主流。研究热点聚焦于动态渲染效率优化 (如 GPU 加速、多线程计算、视频压缩技术)、大屏

高分辨率适配（分布式数据调度、LOD 模型与瓦片化数据组织）及多源数据融合交互（时空过程动态表达、移动端协同操作）。对标量场数据而言，海洋数据量庞大，传统 Web 端的可视化方法在处理大规模网格数据时效率较低，目前多数研究局限于二维平面展示，缺乏有效的三维深度信息表达机制，海面波浪纹理的相关实现较少；对矢量场数据而言，点图标法在数据密集区域易产生箭头重叠和遮挡，可视化效果混乱，粒子系统法在 Web 环境下大规模粒子轨迹计算存在性能瓶颈，CPU 占用率过高。未来，随着 WebGPU、WebAssembly 等高性能计算技术的普及，浏览器端将支持更复杂的体绘制与粒子系统实时模拟，这将为可视化技术的发展打开新的可能性。同时，人工智能技术的进步，将驱动数据的自动分类、异常检测与交互式分析，进一步增强可视化系统的智能化决策能力，推动海洋要素场可视化向实时化、沉浸式与知识发现导向发展。

1.3 研究内容和技术路线

1.3.1 研究内容

本文的研究目标为海洋环境要素场可视化方法研究与系统平台研制，主要研究内容包括以下几个方面：

（1）Web 海洋环境标量场可视化方法研究。研究将基于离散数据实例化颜色映射法、纹理映射法在 Web 端进行大规模网格数据的可视化，在 GPU 端进行数据的插值处理运算，提高渲染效率，在 Web 端基于光线追踪算法实现一种海面波浪生成法。

（2）Web 环境海洋矢量场可视化方法研究。研究基于点图标法，优化布局策略（等间隔采样），减少重叠遮挡问题；基于粒子系统算法，将主要的计算逻辑放置在 GPU 内执行，提高粒子系统渲染的效率。

（3）海洋环境要素场数据处理。研究将基于 NetCDF 数据，研究数据的读取方法以及过滤、采样数据方法。

（4）海洋环境可视化系统搭建。研究将基于 React、Cesium 框架，搭建一个高效、轻量的 NetCDF 数据可视化系统，主要功能用于支持 NetCDF 格式数据可视化、海洋环境标/矢量场的可视化，支持大小屏双端使用、主题切换、语言切换等功能。

1.3.2 技术路线

根据本文研究内容，将技术路线分为三部分，分别为数据处理、可视化方法、系统

搭建，技术路线如图 1-1 所示。

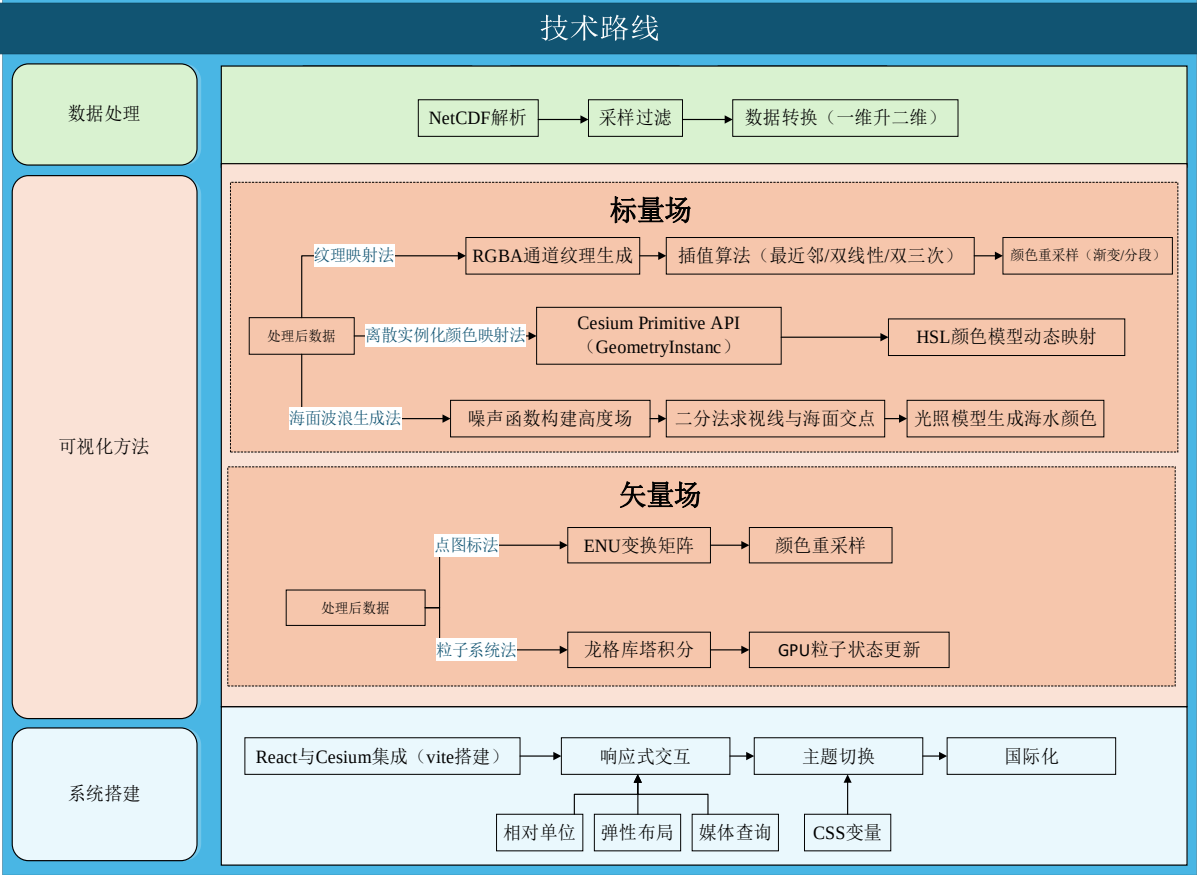


图 1-1 技术路线图

技术路线围绕海洋环境要素场 Web 可视化系统的构建，分为以下三个核心研究内容：

（1）数据处理。研究 NetCDF 格式海洋环境要素场数据的读取、解析与转换方法，解决原始一维数据的二维化重构、数据采样与过滤一级数据服务化等关键问题，为可视化提供高效、规范的数据基础。

（2）可视化方法。研究并设计适用于 Web 环境的海量标量场和矢量场数据的可视化方法。在标量场部分，设计离散数据实例化颜色映射法、纹理映射法以及海面波浪生成法；在矢量场部分，设计点图标法和粒子系统法。

（3）系统搭建。基于 React 前端框架和 Cesium 三维地理引擎，结合 Node.js 后端服务，构建 B/S 架构的海洋环境要素场可视化系统。实现 NetCDF 文件结构查看、数据上传进度显示、标/矢量场可视化方法集成、响应式布局、国际化语言处理、主题切换等核心功能。

1.4 本文组织结构

本文主要内容和章节安排如下所示：

第1章 绪论。阐述海洋环境要素场可视化的研究背景与意义，系统梳理国内外在标量场、矢量场及 Web 可视化领域的研究现状，明确本文研究内容与技术路线。

第2章 海洋环境要素场可视化方法。设计并阐述本文针对 Web 环境设计的海量海洋标量场（如温度、高度）与矢量场（如风场）数据的核心可视化方法体系，包括离散数据实例化颜色映射法、纹理映射法、海面波浪生成法、点图标法及粒子系统法，并详述其底层数学原理与实现机制。

第3章 海洋环境要素场可视化系统实现。介绍基于 React、Cesium 与 Node.js 构建的 B/S 架构可视化平台的整体设计，详细说明 NetCDF 格式数据的处理流程及前后端搭建的过程。同时，详细介绍标量场与矢量场各可视化方法在平台中的具体实现步骤与效果。

第4章 总结与展望。总结并分析本文设计的海洋环境要素场可视化方法与成果，指出当前系统存在的不足，并据此提出未来可行的研究方向与改进工作。

第 2 章 海洋环境要素场可视方法

2.1 海洋环境要素标量场可视化方法

2.1.1 离散数据实例化颜色映射法

离散实例化标量映射法即它通过离散数据实例化一个几何图形，使用颜色或者高度来表达该点位的数值大小，不对数据进行任何的插值。

本研究处理的 NetCDF 格点数据记录了一定分辨率下每个经纬度坐标对应的记录值，Cesium 内部封装实例渲染对象是基于 WebGL 实现的，想要将这些离散数据实例化成几何图形映射到地球表面上，首先需要了解 WebGL 的渲染管线如何进行绘图以及 Cesium 是如何对 WebGL 进行封装的。

(1) WebGL 渲染管线

WebGL 的渲染管线负责将物体 3D 坐标也就是经纬度坐标转变为屏幕空间的 2D 坐标，并且对屏幕中每个像素点进行作色，渲染管线如图 2-1 所示。其中蓝色部分和绿色部分即为 Cesium 暴露给开发者自定义的部分。

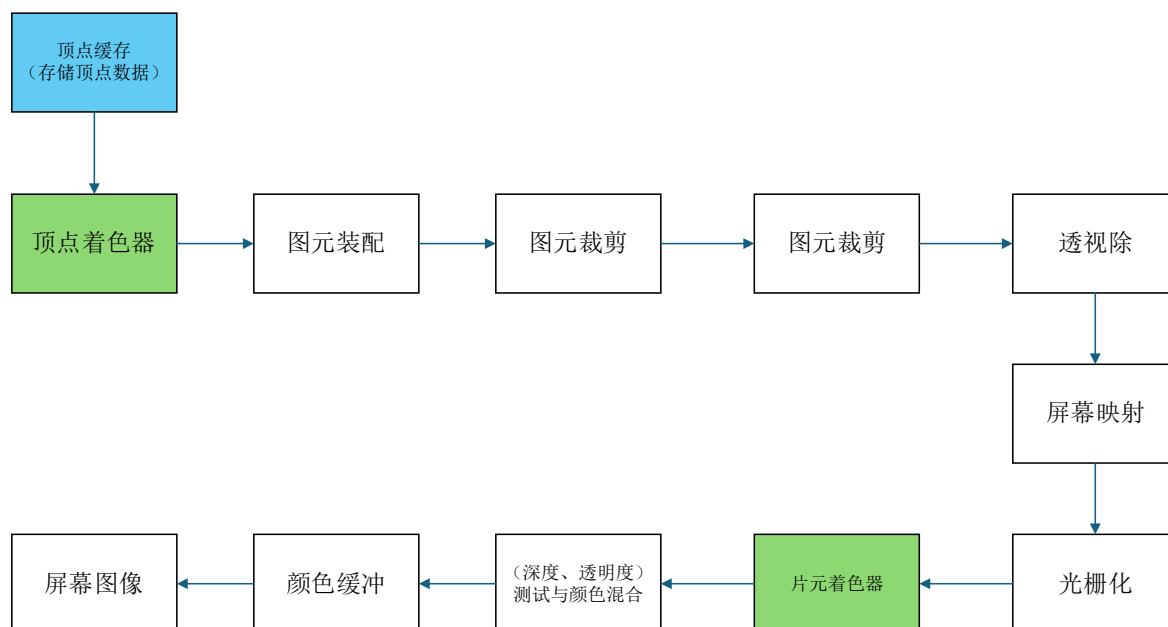


图 2-1 WebGL 渲染管线

通过 HTML5 的 Canvas 元素，开发这可以在网页中动态绘制各种图形、图像和动画，在技术实现层面，Canvas 实际上是在浏览器的渲染主线程上创建了一个强大的操作管线

的总管家对象，这个总管家对象负责管理和调度所有的绘图命令，可以对顶点缓存区中的数据进行精细的调度与控制。

（2）Cesium 的着色器封装

Cesium 提供一系列参数实例化几何图形的 API，其中常用如表 2-1 所示。设置好几何参数后，即可生成几何实例对象，几何实例对象并不能直接渲染到场景上，还需对几何实例图像添加 MaterialAppearance 类（材质外观），Cesium 的 MaterialAppearance 类对顶点着色器和片元着色器进行了一些封装处理，顶点着色器来将模型坐标经过一系列的变换矩阵转化为裁剪空间坐标。片元着色器采用 Phong 光照模型，通过法线、视线方向、光照等信息计算物体表面像素颜色。其流程如图 2-2 所示。其中蓝色块表示输入，绿色块表示各着色器的输出。给 MaterialAppearance 类传入顶点着色器和片元着色器代码，在片元着色器当中自定义代码即可完成几何实例图形颜色的自定义。

表 2-1 Cesium 中常见的参数实例化几何图形 API

几何图形	类名
立方体	BoxGeometry
矩形	RectangleGeometry
圆形	CircleGeometry
椭圆、椭圆柱	EllipseGeometry
多边形	PolygonGeometry
球面	SphereGeometry
墙体	WallGeometry

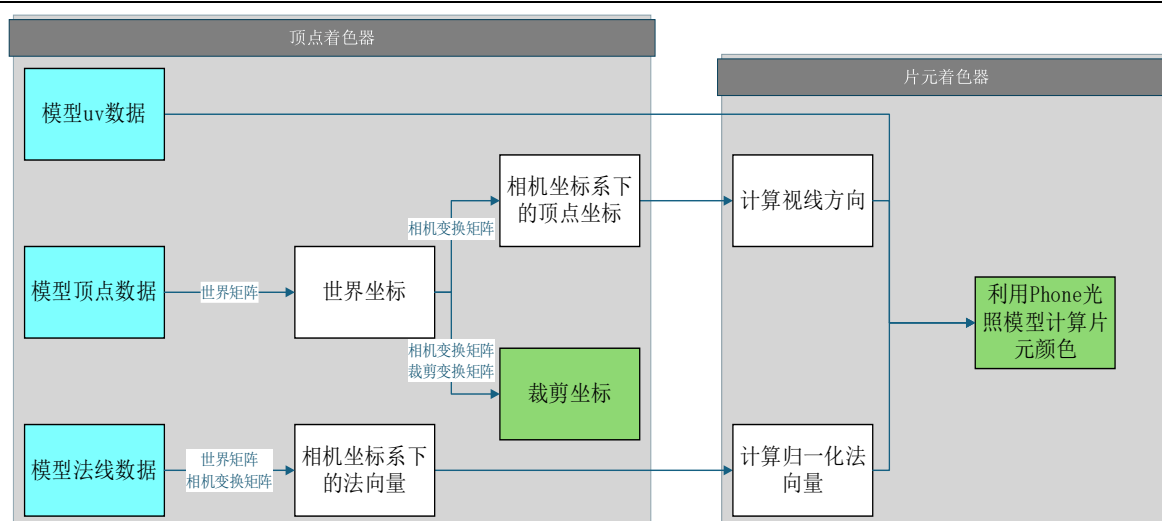


图 2-2 Cesium 的 MaterialAppearance 对象中着色器的计算流程

2.1.2 纹理映射法

纹理映射法是根据 NetCDF 格点数据生成一张 UV(U V parameterization, U 和 V 分

别代表二维纹理坐标的两个轴)纹理图片,这张图片具有 RGBA (Red Green Blue Alpha, 红色、绿色、蓝色、阿尔法)通道,每个通道均可携带一个值,将网格数据的地理位置信息保存在纹理图片的宽高上,将其强度信息保存在 RGBA 其中一个通道上。在地球表面实例化一个包裹全球的几何平面,将纹理图片通过插值法(如最邻近、双线性、双三次)映射到几何体的表面。不同于前文中提到的离散数据实例化颜色映射法,该方法渲染结果是连续的而不是离散的。

(1) 温度纹理图片生成

生成温度纹理图片的重点是将经纬度网格一一映射到纹理图片上,假设网格内某点的经度为 lon , 纬度为 lat , 温度为 $temp$, 其映射方式如公式 (2-1) 所示。

$$\begin{cases} x = \text{round}(\frac{lon - lon_{\min}}{lon_{\max} - lon_{\min}} \cdot W) \\ y = \text{round}(\frac{lat - lat_{\min}}{lat_{\max} - lat_{\min}} \cdot H) \\ R(x, y) = temp(lon, lat) \end{cases} \quad (2-1)$$

其中, x 、 y 分别表示图片的水平、竖直方向像素坐标; $lon_{\max}$ 、 $lon_{\min}$ 分别表示 NetCDF 温度网格的最大经度值和最小经度值; $lat_{\max}$ 、 $lat_{\min}$ 分别表示 NETCDF 温度网格最大纬度值和最小纬度值; W 表示图片的宽度,取 NetCDF 温度网格经度维度; H 表示图片高度,取 NetCDF 温度网格纬度维度; $R(x, y)$ 表示图片上 (x, y) 坐标位置的红色通道数值; round 表示四舍五入。

(2) 颜色纹理创建

由于温度数据是一维的,所以只需要创建一个横向的颜色条作为颜色纹理即可,常见的颜色条创建方式有 2 种:线性渐变颜色条、离散分段颜色条。利用该颜色条,作为纹理图片传递给片元着色器,片元着色器拿到该像素点的温度值,再到颜色条里进行采样,拿到该像素点应当渲染的颜色并输出。

线性渐变颜色条的数学原理是在相邻颜色之间进行 RGB 通道的线性插值,实现平滑颜色的效果,以下以 k 个颜色值为例,每个颜色代表一个温度值,如何在这 k 个颜色值之间进行插值,并计算出某个温度值对应的渐变颜色,对任意温度 T , 其颜色计算公式如公式 (2-2) 所示。

$$\begin{cases} R(T) = R_i + \frac{R_{i+1} - R_i}{\Delta T} \cdot (T - T_i) \\ G(T) = G_i + \frac{G_{i+1} - G_i}{\Delta T} \cdot (T - T_i) \\ B(T) = B_i + \frac{B_{i+1} - B_i}{\Delta T} \cdot (T - T_i) \\ \alpha(T) = \alpha_i \\ \Delta T = T_{i+1} - T_i \end{cases} \quad (2-2)$$

其中, $0 < i \leq k-1$, $T_i < T < T_{i+1}$, $R(T)$ 、 $G(T)$ 、 $B(T)$ 分别代表温度 T 在 R 、 G 、 B 三个通道的颜色值; R_i 、 G_i 、 B_i 表示第 i 个颜色在 R 、 G 、 B 三个通道的颜色值; T_i 表示第 i 个颜色对应的温度。

离散分段颜色条的数学原理是将温度范围 $[T_{\min}, T_{\max}]$ 分为 k 个区间, 每个区间对应固定颜色值, 属于分段常数插值, 每个区间温度边界如公式 (2-3) 所示。

$$T_i = T_{\min} + i \cdot \frac{T_{\max} - T_{\min}}{k} \quad (0 \leq i \leq k) \quad (2-3)$$

对于任意的温度 T , 取其对应的温度边界内的颜色值即可。

(3) 颜色重采样插值

常用的插值方法有最邻近插值法、双线性插值法和双三次插值法等。

最邻近插值法即采用距离自身最近的点的像素值作为自身的像素值, 这是最简单, 计算最快的方法, 可将该点落入的网格区域划分为如图 2-3 所示, 该点落在 A 区、B 区、C 区、D 区分别取左上角、右上角、左下角、右下角像素点的像素值。

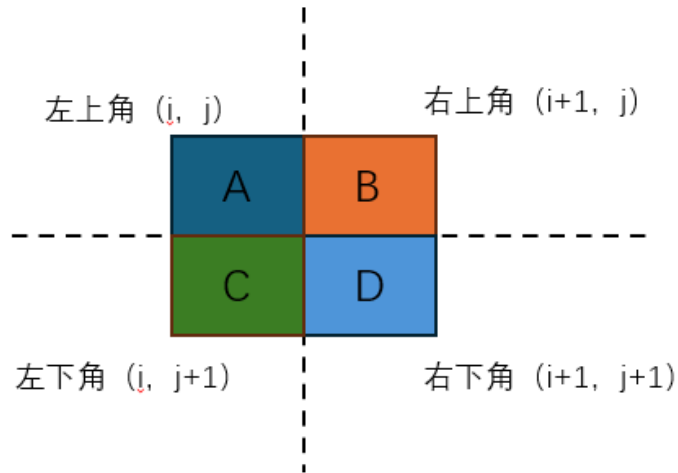


图 2-3 最邻近插值法

双线性插值法则在水平、竖直两个方向分别进行一阶线性插值，假设 P 点为几何图形的 uv 采样坐标， Q_{12} 、 Q_{22} 、 Q_{11} 、 Q_{21} 该采样点周围左上角、右上角、左下角、右下角的温度纹理图片像素点，如图 2-4 所示。该 P 点在水平方向的插值公式如公式 (2-4) 所示，在竖直方向的插值公式如公式 (2-5) 所示，结合公式 (2-4) 和公式 (2-5)，可得到公式 (2-6)，即 P 点的插值结果。

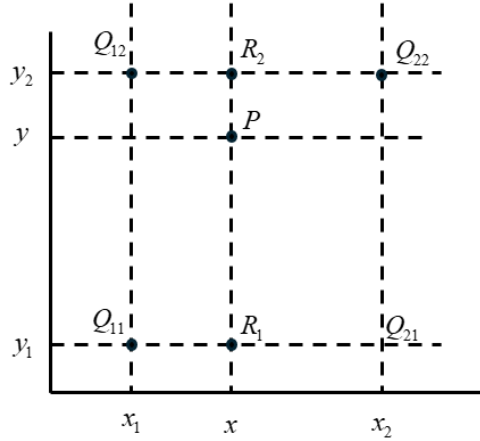


图 2-4 双线性插值

$$\begin{cases} f(R_1) = f(x, y_1) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \\ f(R_2) = f(x, y_2) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}) \end{cases} \quad (2-4)$$

其中， f 函数表示输入点的在纹理图片上的采样结果，其余参数如图 2-4 所示

$$f(x, y) \approx \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2) \quad (2-5)$$

其中， f 函数表示输入点的在纹理图片上的采样结果，其余参数如图 2-4 所示

$$f(x, y) = \frac{y_2 - y}{y_2 - y_1} \left(\frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \right) + \frac{y - y_1}{y_2 - y_1} \left(\frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}) \right) \quad (2-6)$$

双三次插值又称立方卷积插值。三次卷积插值是一种更加复杂的插值方式。该算法利用待采样点周围 16 个点的灰度值作三次插值，不仅考虑到 4 个直接相邻点的灰度影响，而且考虑到各邻点间灰度值变化率的影响。三次运算可以得到更接近高分辨率图像的放大效果。对于给定纹理坐标 $uv \in [0, 1]^2$ 和纹理分辨率 $R(W, H)$ ，其权重函数可由公式 (2-7) 可得，其插值结果由公式 (2-8) 可得。

$$W(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1, & |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a, & 1 \leq |x| \leq 2 \\ 0 & \end{cases} \quad (2-7)$$

其中, a 通常取-0.5 和-1, 当 a 取-0.5 时, 该权重函数更注重平滑性, 取-1 时, 更强调边缘锐化效果。 x 表示两个像素之间的距离。

$$\begin{cases} vc = [uv \cdot R] \cdot \frac{1}{R} \\ f = fract(uv \cdot R) \\ fract(t) = t - \lfloor t \rfloor \\ I_{out}(uv) = \sum_{i=-1}^2 \sum_{j=-1}^2 I_{in} \left(vc + \frac{(i, j)}{R} \right) \cdot W(f_x - i) \cdot W(f_y - j) \end{cases} \quad (2-8)$$

其中, uv 表示采样点的 uv 坐标; R 表示纹理图片的参数, 在水平方向插值时, 表示图片的宽, 在垂直方向插值时, 表示图片的高; vc 表示插值左上角的纹理取样值; $I(uv)$ 表示在在 uv 坐标的采样结果; I_{out} 表示插值结果, I_{in} 表示图片上对应的纹理值。

2.1.3 海面波浪生成法

在海洋环境要素场的可视化过程中, 常常需要对海面进行渲染, 海面渲染的关键有两点, 一是要生成波浪噪声, 二是使用光线追踪算法计算海面颜色。波浪噪声既要有随机性, 也要有规律性, 规律性体现在水平方向是连续的。光线追踪算法的总体的思路是, 根据每个像素点代表的海平面高度, 计算出其法线向量, 根据入射光和反射光进行一定的比例混合叠加 (这个比例根据菲涅尔效应计算), 加上镜面高光分量和距离衰减分量, 渲染出该像素的颜色值。

为了逼近真实的渲染效果, 某个片元中视线与与海平面的交点不应该是一个固定的值, 而是考虑到海面高度的变化, 可采用二分法步进来逐步逼近光线与海面的交点, 根据海浪高度与海平面高度的差值对像素颜色进行增强处理, 模拟真实世界中海域深浅的效果。根据视线距离给颜色设置衰减效果。

(1) 旋转矩阵

在实现海面波浪生成法, 需要设置视线的向量, 视线的向量主要是由屏幕坐标生成

的，屏幕坐标的范围为 $[-1,1]$ ，在负值部分，生成的视线向量将方向指向天空，而我们希望视线的向量一直指向海平面，所以需要对视线的向量进行旋转。对于任意一点 $P(x,y,z)$ ，绕 x 轴旋转 α ， y 轴旋转 β ， z 轴旋转 γ ，其新坐标可有公式（2-9）得出。

$$\begin{aligned} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} &= \begin{bmatrix} \cos \gamma & \sin \gamma & 0 \\ -\sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & \sin \beta \\ 0 & -\sin \beta & \cos \beta \end{bmatrix} \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \\ &= \begin{bmatrix} \cos \alpha \cos \gamma - \cos \beta \sin \alpha \sin \gamma & \sin \alpha \cos \gamma + \cos \beta \cos \alpha \sin \gamma & \sin \beta \sin \gamma \\ -\cos \alpha \sin \gamma - \cos \beta \sin \alpha \cos \gamma & -\sin \alpha \sin \gamma + \cos \beta \cos \alpha \cos \gamma & \sin \beta \cos \gamma \\ \sin \beta \sin \alpha & -\sin \beta \cos \alpha & \cos \beta \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \end{aligned} \quad (2-9)$$

其中， X ， Y ， Z 表示原坐标 x ， y ， z 旋转后得到的新坐标。

（2）构建海浪高度场

构建海浪高度场首先需要构建一个噪声函数对波形进行扰动，然后构建单尺度波形函数计算出海浪的高度。假设视线向量与海平面的交点坐标为 $p(x,y)$ ，将其坐标分解为整数部分 $\lfloor p \rfloor = (i, j)$ 和小数部分 $f = (f_x, f_y) = p - \lfloor p \rfloor$ ，该点的噪声生成可参考公式（2-10）。此公式使用的插值公式在 C^1 （一阶导数）连续，故生成的噪声结果比较平滑。生成的噪声在 x 方向上具有方向连续性：固定 y 时， $n_x^{(j)}$ 和 $n_x^{(j+1)}$ 均为 f_x 的 C^1 函数。生成噪声函数之后，需要应用到单尺度波形函数上，单尺度波形函数有公式（2-11）计算可得。利用单尺度函数可以生成该点的海浪高度，然而单尺度计算海浪高度并不能真实的模拟自然界中的海浪，自然界中的海浪的传播受到多种因素的影响。

$$\left\{ \begin{aligned} fract(t) &= t - \lfloor t \rfloor \\ h(p) &= fract[\sin(127.1p_x + 311.7p_y)] \\ w_x(f_x) &= 3f_x^2 - 2f_x^3 \\ w_y(f_y) &= 3f_y^2 - 2f_y^3 \\ n_x^{(j)}(f_x) &= h(i, j)(1 - w_x) + h(i + 1, j)w_x \\ n_x^{(j+1)}(f_x) &= h(i, j + 1)(1 - w_x) + h(i + 1, j + 1)w_x \\ n(p) &= n_x^{(j)}(1 - w_y) + n_x^{(j+1)}w_y \end{aligned} \right. \quad (2-10)$$

其中， w_x 为水平插值权重； w_y 为竖直插值权重； $n(p)$ 即为 p 的噪声生成结果。

$$\left\{ \begin{array}{l} S(uv, C) = \left[1 - \left(\prod_{k \in \{x, y\}} (1 - |\sin(uv)|)^{\alpha_k} \cdot |\cos(uv)|^{\beta_k} \right)^{0.65} \right]^C \\ uv = uv + \lambda n(uv) \\ a_k = 1 - \theta(|\sin(uv)| - 0.5) \\ \beta_k = \theta(|\sin(uv)| - 0.5) \end{array} \right. \quad (2-11)$$

其中 uv 可用视线向量与海平面的交点坐标 $p(x, y)$; S 表示该点计算的海浪高度; n 即为公式 (2-10) 中的噪声函数; λ 为噪声强度; θ 为 Heaviside 阶跃函数。

(3) 二分法计算光线与海面交点

光线迭代行进的过程中, 海浪的高度也在随之发生变化, 可以用二分法来模拟光线的行进, 经过一定次数的迭代, 光线将与海平面进行相交, 在实际的计算当中, 一般将迭代次数设置为固定的值, 提高渲染效率。设置光线参数方程如公式 (2-12) 所示, 海面高度场函数定义如公式 (2-13) 所示, 其等式为 0 意味着光线步进的结果点 y 坐标与海面高度重合。可使用二分迭代的方法近似求解该方程, 在每次迭代的过程中, 海浪高度根据公式 (2-11) 重新计算, 模拟现实中光线逐渐逼近的场景。在初始时刻, 定义 t_0 为 0, t_1 为 1000, 将 t_1 公式应用到公式 (2-12), 再将其结果应用到公式 (2-13)。若得到的结果大于 0, 说明在 $t \in [t_0, t_1]$ 范围内, 光线未到达海平面, 故无需迭代, 无法求解, 取 t_1 作为近似海平面点即可; 若得到的结果小于 0, 说明在 t_1 时刻, 光线已经越过海面进入海面内部, 需要使用二分法求解其与海平面的交点。迭代公式如公式 (2-14) 所示, 迭代次数可自行选择, 迭代次数越高精度越高, 但渲染性能更差, 需要根据实际情况取舍。

$$P(t) = O + t \cdot d, \quad t \in [t_0, t_1] \quad (2-12)$$

其中, O 为光线起点, 也就是眼睛坐标; d 为归一化视线方向向量; t_0 , t_1 为初始搜索区间的断点。

$$f(P(t)) = y(t) - H(x(t), z(t)) \quad (2-13)$$

其中, $y(t)$ 表示光线在 t 处的 y 坐标; H 为波浪高度函数, 其可由公式 (2-11) 计算得到;

$$\begin{cases} t^{(k)} = t_0^{(k-1)} + \frac{f(t_0^{(k-1)})}{f(t_0^{(k-1)}) - f(t_1^{(k-1)})} (t_1^{(k-1)} - t_0^{(k-1)}) \\ f(t^{(k)}) = y(t^{(k)}) - H(x(t^{(k)}), z(t^{(k)})) \end{cases} \quad (2-14)$$

其中, $t^{(k)}$ 表示第 k 次迭代的中间参数, 若 $f(t^{(k)}) < 0$, 则 $t_1^{(k)} = t^{(k)}$, 否则 $t_0^{(k)} = t^{(k)}$ 。

(4) 海面法线计算

前文中, 已经实现了海平面高度的计算, 接下来需要利用这个海平面高度计算法线, 便于后续计算海水光线融合模型。海面法线可通过中心差分法近似高度场梯度, 其数学表达式为公式 (2-15) 所示。当海面高度远大于海面基准高度时, 法线的梯度变化如公式 (2-16) 所示, 即法线的 x 方向和 z 方向分量都将趋近于 0, 法线的 y 方向将占主导地位, 法线方向近似与垂直水面向上, 反射效果强, 逼近真实生活中的海面场景。

$$n = \frac{1}{\|\nabla f(p)\|} \begin{bmatrix} f(p_x + \epsilon, p_y, p_z) - f(p) \\ \epsilon \\ f(p_x, p_y, p_z + \epsilon) - f(p) \end{bmatrix} \quad (2-15)$$

其中, $f(\mathbf{p})$ 为海面高度场函数, 如公式 (2-10) 所示。

$$\begin{cases} \frac{\partial f}{\partial x} = \frac{\partial}{\partial x} (p_y - H) = -\frac{\partial H}{\partial x} \rightarrow 0 \\ \frac{\partial f}{\partial z} = -\frac{\partial H}{\partial z} \rightarrow 0 \end{cases} \quad (2-16)$$

其中, p_y 为公式 (2-11) 计算的海面高度, H 为海面基准高度。

(5) 海水颜色生成模型

在片元着色器上, 已知海面的法向量, 此时在场景中添加光源, 给海面设置海水底色, 反射光以及天空颜色, 就可以利用海水颜色生成模型渲染出该片段海面的渲染颜色。其海水颜色生成模型参考公式 (2-17)。海水颜色由 4 部分光强分量合成, 分别是折射分量 C_{refr} , 反射颜色分量 C_{refl} , 镜面高光分量 C_{spec} , 高度衰减分量 C_{height} , 使用结合菲涅尔效应给反射光设置权重, 入射角越大, 反射光强越强, 根据眼睛位置和视线与海平面交点的距离; 根据法线、光源、眼睛位置添加高光效果, 增强视觉真实感; 计算颜色衰减, 模拟距离对颜色的影响, 即远处的海水颜色更暗。

$$\left\{ \begin{array}{l} C_{\text{sea}} = (1-F)C_{\text{refr}} + FC_{\text{refl}} + C_{\text{spec}} + C_{\text{height}} \\ F = \kappa \cdot (1-n \cdot (-v))^\gamma \\ C_{\text{refr}} = c_{\text{base}} + K_d \cdot c_{\text{water}} \\ C_{\text{refl}} = C_{\text{sky}} \\ C_{\text{spec}} = \eta \cdot (r_{\text{spec}} \cdot l)^\beta \\ r_{\text{spec}} = 2(n \cdot h)n - h \\ h = \frac{v+l}{\|v+l\|} \\ C_{\text{height}} = \xi \cdot (y-y_0) \cdot c_{\text{water}} \cdot (1-\|d\|^2 \cdot 0.001) \end{array} \right. \quad (2-17)$$

其中, C_{sea} 表示海水渲染颜色, C_{refr} 表示海面折射分量, C_{refl} 表示海面反射分量, C_{spec} 表示海面镜面高光分量, C_{height} 表示高度衰减分量; κ 表示菲涅尔系数缩放因子, n 表示此处海面的法向量, γ 表示菲涅尔衰减系数; c_{base} 表示海水基底色, c_{water} 表示水体反光色, K_d 表示漫反射系数, 可取 $K_d = (n \cdot l \cdot 0.4 + 0.6)^{80}$, l 表示光源方向; C_{sky} 表示天空颜色; η 表示能量守恒归一化因子, 可取 $\eta = \frac{\beta+8}{8\pi}$, β 表示高光衰减指数; r_{spec} 表示 Blinn-Phong 半角向量, v 表示视线方向; ξ 表示高度影响系数, y 表示当前海面点的高度, y_0 表示基准海面高度, d 表示视线起点到海平面交点的向量。

2.2 海洋环境要素矢量场可视化方法

2.2.1 点图标法

点图标法是矢量场可视化中被广泛使用的经典方法, 其核心思想是在离散空间的每个点位放置离散图标直观反映矢量场的特征, 每个图标的位置对应数据点的坐标, 图标的方向对应数据点的方向, 图标的颜色表示了该点矢量的强度。

点图标法的渲染方法与前文中表两场的渲染方式——离散数据实例化颜色映射法差别不大, 都是在地球网格的每个点上实例化一个几何图形, 根据该点的数值赋予对应的颜色。

点图标渲染法的核心有两个, 一个是根据点的 UV 风速设置图标的旋转角度, 其旋转矩阵参考前文中的公式 (2-9), 二是创建颜色条纹理, 根据风速大小为每个图标着色,

颜色纹理公式的生成参考前文的公式(2-1)和公式(2-2)，与标量场不同，标量场的数值只涉及单一数值，矢量的数值由UV两个方向组成，值可以通过风的两个方向向量U、V的向量模获得，用其模来代表矢量强度。

2.2.2 粒子系统法

粒子系统法是利用大量粒子运动来模拟现实生活中的物理现象，常用于流场、风场、喷雾、烟花的可视化。粒子系统的最小单元是粒子，每一个粒子都有属于自己的生命周期，粒子系统的生命周期分为“生成”、“运动”、“消亡”，实现这三个生命周期，就能实现粒子系统的渲染。在“生成”阶段，首先指定一篇区域，在该区域内随机生成一定数目的粒子；在“运动”阶段，结合NetCDF数据文件提供的速度信息，经过双线性插值后获取当前位置的速度，再采用龙格库塔算法来模拟计算粒子的运动速度；在“消亡”阶段，需要在指定区域内重新生成粒子，保证总粒子数不变，粒子的消亡条件是超出边界或者触发随机掉落概率，可根据速度调整掉落概率，运动的越快的粒子越容易进入“消亡”阶段。

粒子的运动速度可用龙格库塔方法计算。龙格库塔方法是数值积分中应用最广泛且效果显著的技术之一。它的原理基于泰勒级数展开，充分利用级数的高精度特性，有效避免了高阶导数计算的难题。该方法的核心思想是通过点的选择（点定位）迅速确定粒子在下一时刻的位置，从而提升数值积分的精度并降低误差。其二阶龙格库塔法一般形式见式(2-18):

$$\begin{cases} k_1 = f(t_n, y_n) \\ k_2 = f(t_n + h, y_n + k_1 h) \\ y_{n+1} = \frac{h}{2} f(k_1 + k_2) \end{cases} \quad (2-18)$$

其中 k_1 和 k_2 代表斜率， h 是数值积分步长。

第3章 海洋环境要素场可视化系统实现

3.1 系统实验环境

本系统搭建所用到的软硬件环境如表 3-1 所示。

表 3-1 硬件设备及其软件环境

实验环境	
个人计算机和计算服务器	
硬件配置	CPU: 12th Gen Intel(R) Core(TM) i7-12700H, 2300 Mhz, 14 个内核, 20 个逻辑处理器
	RAM: 32GB
	GPU: Intel Iris Xe Graphics
	显存: 1GB
软件环境	操作系统: Windows 11 Home China (64 位)
	编程语言: JavaScript、GLSL
	平台环境: IntelliJ IDEA 2024.3.5
	开发引擎: React、Cesium

3.2 可视化系统架构设计

3.2.1 系统总体架构

海洋数据种类多、数据量大，本系统基于 NetCDF 格式文件的数据进行可视化，本研究涉及的可视化方法比较多，方法分为标量场和矢量场的可视化，细分领域海洋离散数据实例化颜色映射法、纹理映射法、海面波浪生成法、点图标法、粒子系统法，很适合模块化和组件化。

模块化开发是将程序代码划分为独立的模块，每个模块都有自己的功能和接口，作为同一种类型下的方法，可以单独封装成接口供使用，从而提高代码的可读性，方便代码的迭代维护更新，本系统中的标量场和矢量场分别可以作为一个模块，并且作为一个单独的页面，每个模块负责管理各自渲染方法。

组件化是将一个页面划分为多个组件，组件化是为了提高代码的可读性，减少代码空间体积，方便代码的迭代更新。本系统中颜色图例、NetCDF 文件数据可视化均可以封装成一个组件，无论是标量场还是矢量场页面，都可以直接调用。

本系统采用 B/S 架构，架构设计包含三层，即数据层、中间层以及用户层。系统架构如图 3-1 所示。

分为前端和后端两部分，前端负责可视化，后端负责数据处理计算。

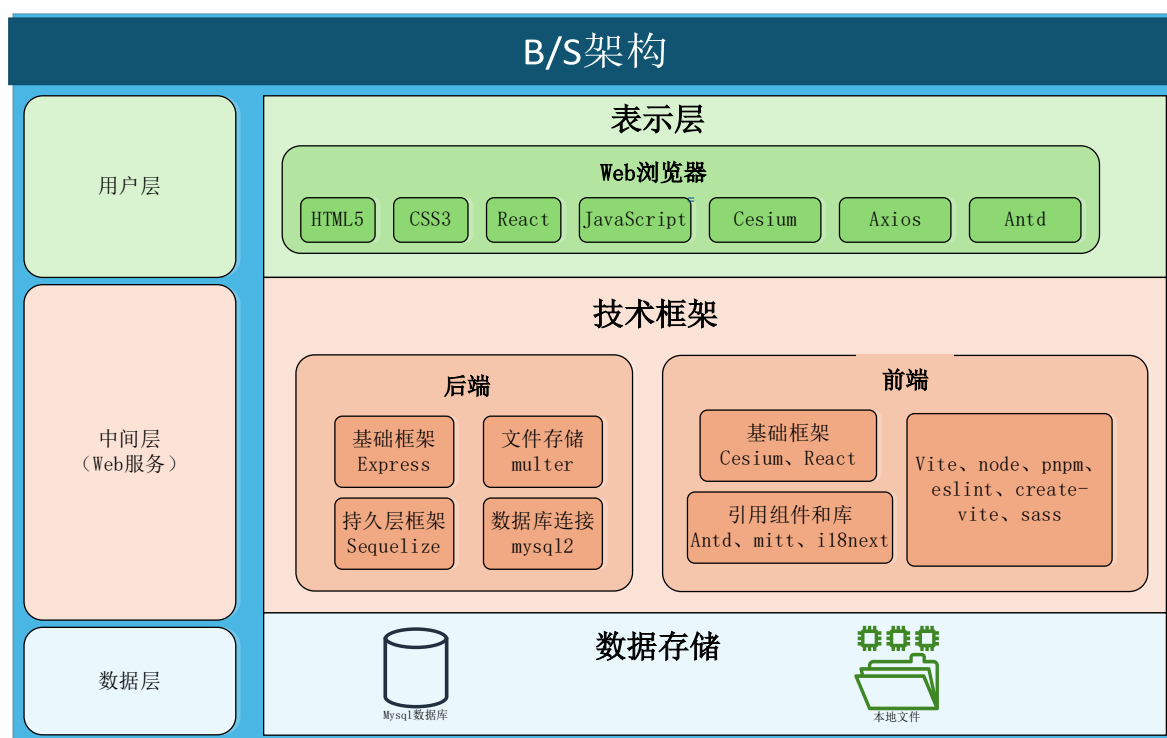


图 3-1 系统总体架构图

数据层为 MySQL 数据库以及本地存储文件，数据库负责记录文件的路劲、信息等，本地文件夹负责记录前端上传的 NetCDF 文件。

中间层主要是 Web 服务的技术框架，技术框架描述前后端使用到的库以及插件、构建工具。

用户层主要是 Web 浏览器，通过 HTML 构建页面，CSS（Cascading Style Sheets，层叠样式表）控制样式、Javascript 控制交互，其余的 React、Cesium、Axios、Antd 等则是基于 Javascript 的二次封装，方便进行快速开发。

3.2.2 前端系统设计

前端基于 React v19.0.0 和 Cesium v1.128.0 可视化框架开发，基于 vite v6.2.4 构建工具进行模块构建以及打包。通过将界面拆分为多个功能组件（如矢量场可视化、标量场可视化、图例展示），实现模块化设计和代码的服用，组件的状态与行为由 React 的状态管理和生命周期机制进行管理和控制，在页面销毁阶段，需要移除该场景下的可视化效果。数据的可视化页面采用 Javascript XML（Extensible Markup Language，可扩展标记语言）进行构建，数据处理算法层面使用 JavaScript 语言进行处理，数据管理层面采用 Cesium 的 Primitive API 进行管理。

本系统统一采用 NetCDF 文件格式处理，而未来可能会增加别的数据格式比如 Grid、

GeoJSON 格式，所以给 NetCDF 文件分配了一个单独的路由，在其之后划分为标量场可视化和矢量场可视化，如图 3-2 所示。

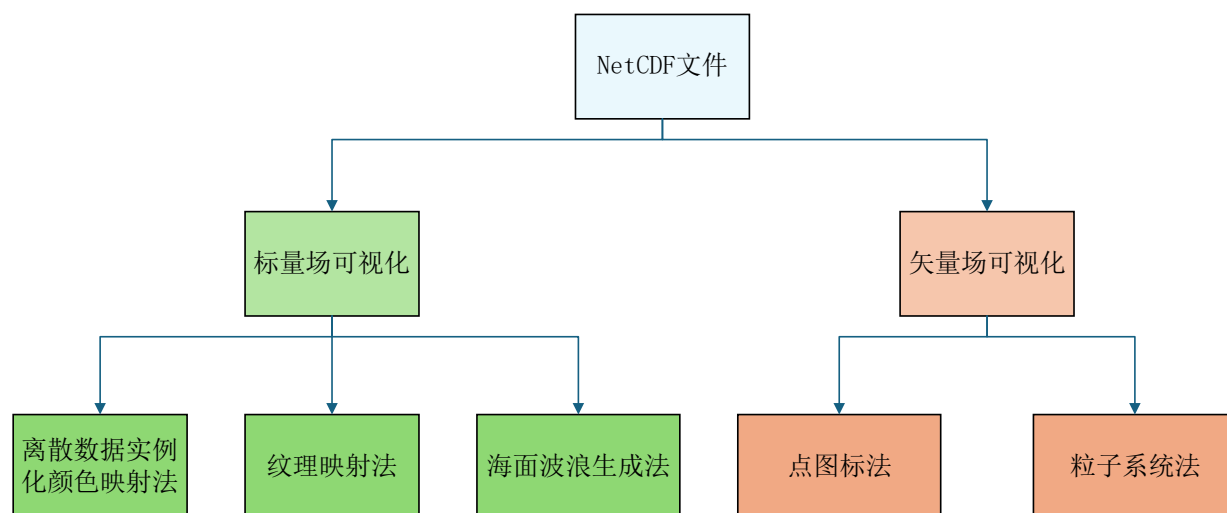


图 3-2 前端路由配置

(1) 页面布局——响应式布局

在以往的网页开发过程中，网页的宽高通常是设定一个固定的值，所以一个网页应用通常要开发手机端和电脑端两个版本，这样的开发路线极大的降低了开发效率。随着 CSS 技术以及网页系统的更新迭代，实现网页布局的响应式成为可能，常用的响应式布局方法为以及各自的优劣如表 3-2 所示。

表 3-2 响应式布局各实现方法对比

响应式布局方法	介绍	优点	缺点
媒体查询	使用 CSS 的媒体查询根据不同的屏幕尺寸调整样式	能够完成任何复杂响应式布局	实现繁琐，需要配合其他布局方法使用，增加项目的维护难度
弹性布局	使用 Flex 盒子布局模型实现元素的弹性排布	简单易用，实现一维元素的排列方式	浏览器兼容性问题，旧版本不支持
相对单位	使用相对单位定义页面元素的	简单直接，可读性高	适用简单场景，缺乏更复杂的响应机制。

本系统的电脑端的页面的标量场可视化页面和矢量场可视化页面图 3-3 所示。两者差别不大，矢量场页面不同于标量场，需要输入 UV 两个变量对应的字段名。电脑端页面由两部分组成，一部分是顶部区域，另一部分是主体区域。顶部区域和主体区域都使用了表 3-2 介绍的 3 种响应式布局方法进行布局，其布局方式能够随页面宽度大小发生变化而变化，本系统采用的判断手机端和电脑端的方法是判断页面的宽度，当页面宽度小于 900 像素时，认定为手机端，使用手机端的布局方式。

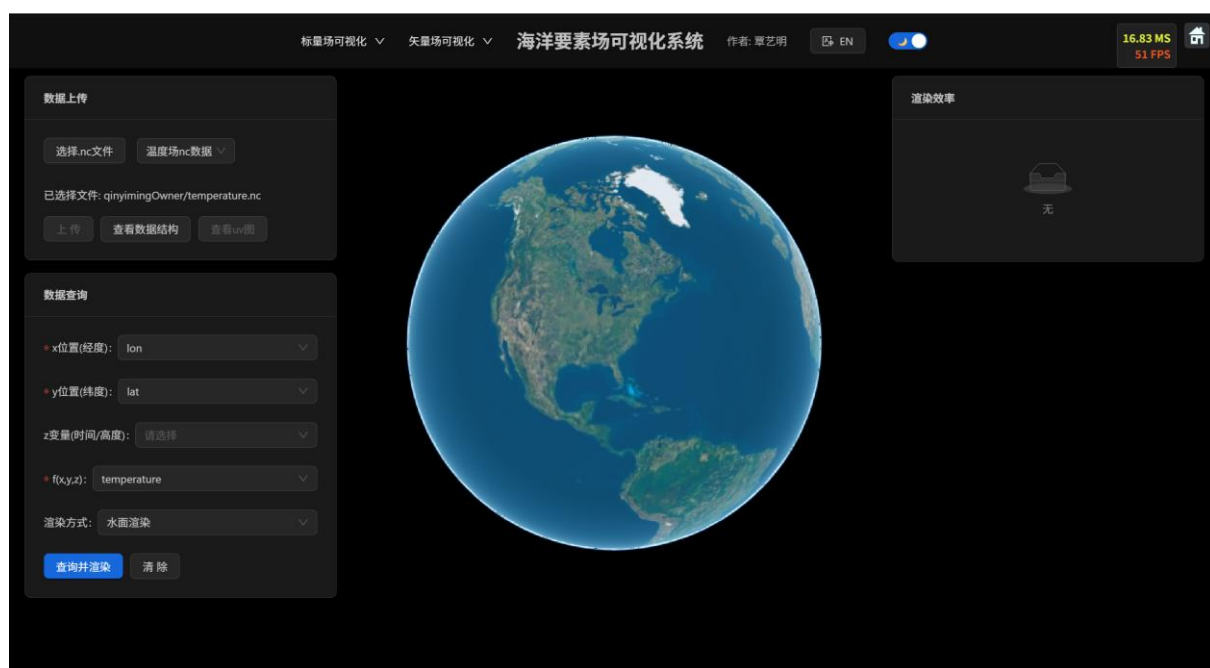


图 3-3 标量场可视化页面

顶部区域由标量场可视化、矢量场可视化、海洋要素场可视化系统、作者、中英文切换、主题切换、帧率显示、返回地球主页总共 8 个部分组成。如果在手机端显示这个页面，无论如何调整，都是不能够完全放下的，所以在手机端，顶部只保留系统标题，帧率显示，将顶部的其他部分放到侧边栏，该侧边栏可折叠可展开，技术原理是预先写好侧边栏以及展开图标的 DOM 结构，给 DOM 注册浏览器点击事件，初始打开页面时侧边栏的显示状态设置为隐藏，当触发点击事件时，设置侧边栏显示，使用 CSS 的媒体查询，查询页面的宽度，当页面的宽度小于阈值时，才将其 DOM 结构设置为显示，否则设置为隐藏。实现效果如图 3-4 中左图所示，左上角框选处即为展开图标，控制侧边栏的展开和隐藏。

主体区域由三个内容卡片（数据上传、数据查询、渲染效率）组成，考虑到手机端屏幕比较小，渲染三个卡片的话，无法平衡空间和清晰度之间的取舍问题，所以在手机端，本系统选择不显示“渲染效率”这个卡片，该卡片的显示与否与系统的可视化效果无关，而其他两个卡片是与标/矢量的可视化密切相关的，所以在手机端，左侧只显示“数据上传”卡片，右侧显示“数据查询”卡片，舍弃掉“渲染卡片”的渲染。技术原理是采用 CSS 的媒体查询，当页面宽度小于阈值时，“渲染效率”卡片隐藏显示，改变“数据上传”、“数据查询”共同容器内部的排列方式，将按列排序改为两端排序，同时将字体的大小缩小，效果如图 3-4 中图所示。在主体区域，有一个按钮“查看数据结构”按

钮，点击该按钮会弹出一个表格页面，该页面记录了 NetCDF 文件数据的头文件信息，该页面同样使用响应式布局，做法与顶部区域类似，做一个折叠图标，如图 3-4 右图所示红框区域。

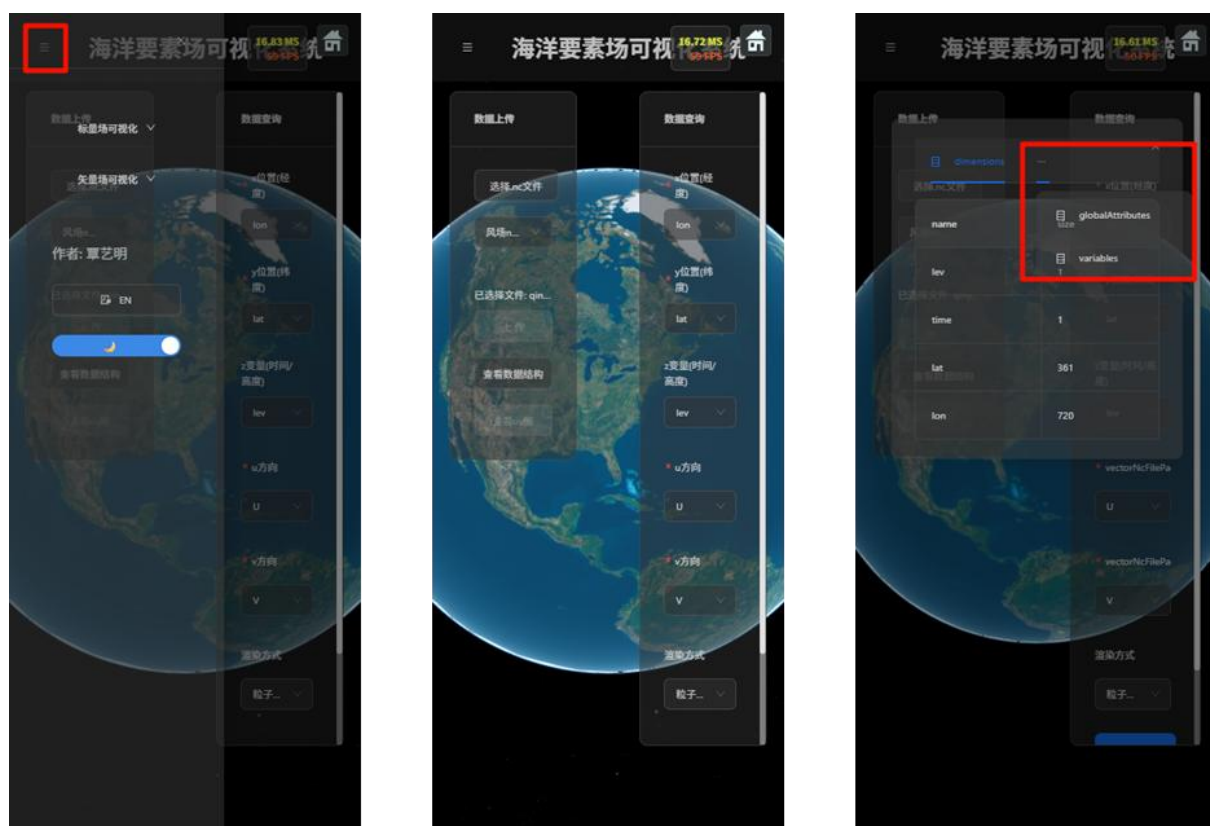


图 3-4 手机端页面

(2) 国际化

国际化（Internationalization，简称 i18n）是指对软件进行设计和开发，使其可以方便地适应不同语言、地区和文化的过程。在 Web 应用中，通常要实现多语言支持、本地化格式（数字、日期）、文本方向（左到右、右到左）。本系统中实现了中文、英文两种语言的切换，方便不同地区的人使用本系统。在前端开发中，实现中英文国际化的常用方法是定义两个 JSON 文件，两个 JSON 文件分别记录中文和英文的翻译，渲染字体的时候，获取当前环境下的语言环境，根据语言环境决定读取哪个 JSON 文件到当前上下文，最终返回翻译字体，使用 React 的调度系统（Fiber 架构）检测语言环境的变化，当语言环境变化时，触发页面的重新渲染。在本系统借助了 i18next 库来完成国际化，其底层实现原理与上述差别不大，底层原理是使用 React 的事件订阅机制，在语言环境改变时，替换配置对象，通知所有使用其配置对象的组件进行状态更新，这个状态的更新是由 React 的 Fiber 架构进行调度完成的。在中文环境下，本系统点击国际化按钮（顶

部区域)后,页面将会刷新显示英文,如图3-5所示。

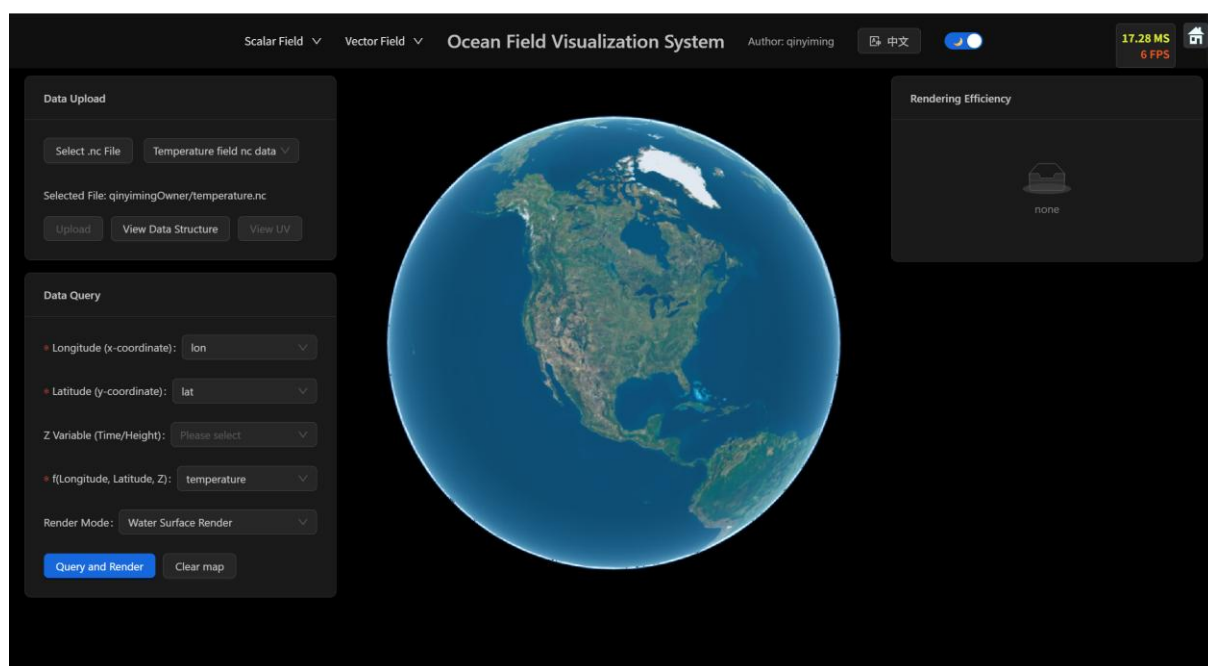


图 3-5 国际化页面(英文)

(3) 主题切换

现代前端网站多具备有主题切换的功能,常规的前端主题切换主要有3种方法,第一种方案是提前准备几套CSS样式主题文件,选择不同的主题,就挂载不同的CSS文件到页面上。第二种提前引入所有主题样式,做类名切换,也就是说率先写好了不同模式下的各类名标签的样式,在切换主题的时候,动态更改类名就行。第三种是使用CSS变量的方式,应用样式的时候,使用变量而不是固定的常量,切换主题的时候,更改对应变量的值。第一种和第二种方式都比较繁琐,工作量大并且不利于项目的更新迭代,故本系统采用第三种方式实现主题切换。

本系统实现了两套样式主题的切换:暗黑模式(默认)、白天模式。并且使用HTML5的localStorage API实现主题模式持久存储记录选择的模式,使得页面刷新情况下主题模式保持原设置模式。暗黑模式下,前端页面背景偏黑,字体偏白;白天模式下,前端页面背景偏白,文字偏黑。此外,本系统在传统的主题切换上做了一个改进:在主题切换的时候Cesium的天空盒背景也对应改变。Cesium的地球背景色默认是纯黑色,用于模拟太空,很显然,纯黑色不适合本系统的白天模式,故本系统在切换为白天模式的时候替换Cesium的纯黑天空盒为一个偏白的天空。效果如图3-6所示。



图 3-6 系统白天主题模式页面

(4) 上传进度条

本系统接收 NetCDF 格式数据，NetCDF 格式的数据一般比较大，Web 环境传输文件数据受前端和后端的网络波动影响都很大，所以上传数据一般需要等待一段时间，故需要将上传进度进行可视化，在前端层面，实现进度条的可视化需要使用 JavaScript 语言控制进度条的宽度，与上传进度进行绑定。在进度获取层面，一般可采用 XMLHttpRequest 对象或者 Axios 库，XMLHttpRequest 是绑定在浏览器上下文环境中的一个类，可以直接使用，使用此类发送请求可在其回调函数中获取上传进度，本系统采用的是 Axios 库，Axios 底层也是基于 XMLHttpRequest 对象实现的，需要注意的是，此进度只能获取近似的上传进度，并不能获取完全真实的上传进度，造成这个原因与 TCP（Transmission Control Protocol，传输控制协议）协议发送数据的方式有关，在客户端，会有一个数据缓存区，这个数据缓存区存储的就是等待发送的数据，TCP 协议层会在适当的时候选取一部分数据发送出去，当上传文件的时候，数据会不断的写入缓存区，每次写入就会被监听到，然后触发 XMLHttpRequest 的上传进度回调，此时前端就会在回调函数中获取进度更新进度条，但是此时数据可能还没有发送出去，仍然被放置在数据缓存区，加上发送数据也需要时间，所以与实际的上传进度是存在一定的误差的。本系统上传的进度条效果如图 3-7 所示

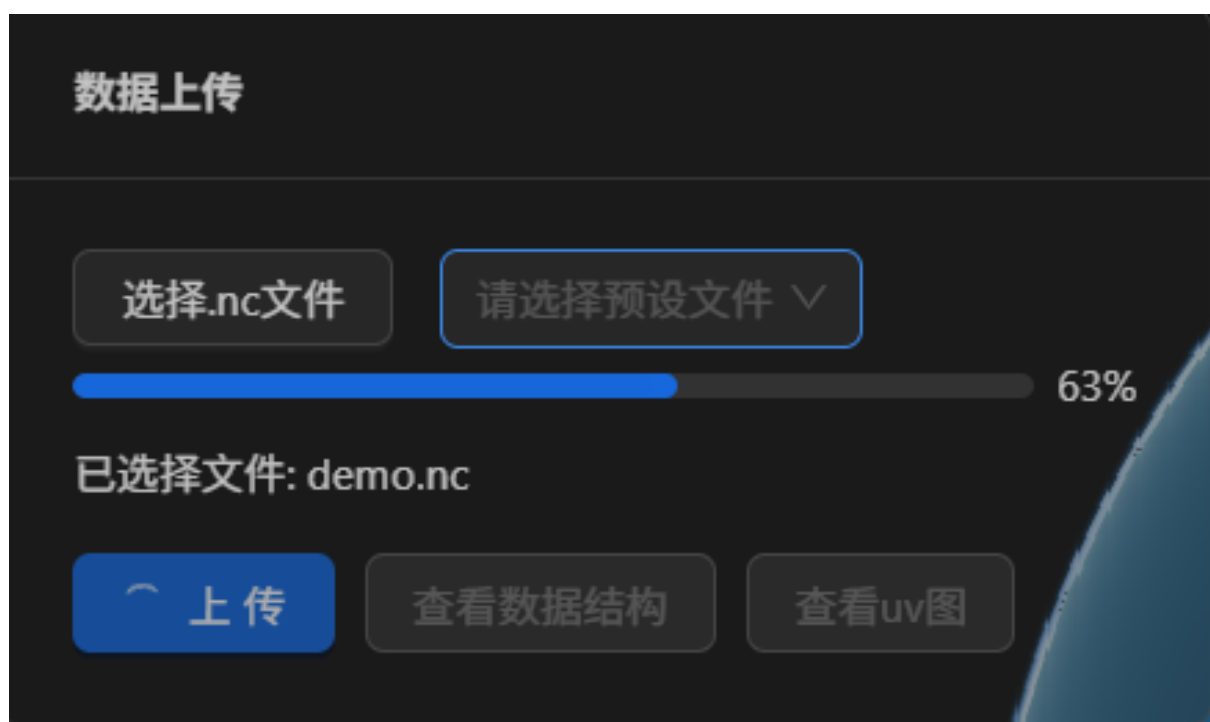


图 3-7 系统 NETCDF 文件上传进度条

3.2.3 后端服务设计

本系统基于 Node.js 的 Express.js v4.18.2 进行后端服务的搭建，Express.js 为开发可扩展的后端应用程序提供了强大的功能和根据，其为 Web 应用、HTTP（HyperText Transfer Protocol，超文本传输协议）请求和相应、路由和中间件提供了一套工具，在 JavaScript 生态系统中被广泛使用。

本系统使用后端服务的有三个原因，一是压缩前端代码体积，二是增强可视化的效率，三是提供系统内置存储文件查看可视化效果。由于浏览器中的 JavaScript 代码是在渲染主线程上执行的，执行 JavaScript 代码的时候会阻塞页面的渲染，所以算法的计算会大大地降低渲染性能，提供后端接口服务，使得算法的复杂计算由三方服务器承担，使得浏览器的渲染压力大大减轻。

本系统的后端服务主要用于 NetCDF 文件的数据处理、NetCDF 文件的校验和 NetCDF 文件的存储，对于实际渲染方面的算法，交给前端系统来进行的。主要通过 RESTful API 实现数据的上传、查询。

对于前端上传过来的 NetCDF 文件，后端首先做一个文件类型校验，确保数据文件的格式是 NetCDF 文件类型的。后端服务处理好的数据，均已 JSON 的格式返回给前端，在 JSON 格式中，有一个 HTTP 响应码，其值以及意义如表 3-3 所示，前端根据响应码

的类型做不同的处理，若响应码为 2 开头，则利用已写好的渲染算法处理后端返回的数据，否则弹出一个错误提示，提示错误的信息。

表 3-3 HTTP 响应码分量及描述

分类	分类描述
1	服务器收到请求，需要请求者继续操作
2	服务器成功处理数据并返回
3	重定向，需要进一步操作以完成请求
4	客户端错误，请求包含语法错误或无法完成请求
5	服务器错误，服务器在处理请求过程中发生了错误

本系统主要实现了 2 个接口，其接口描述如表 3-4 所示。其中，POST 请求的接口主要是用来接收前端上传的 NetCDF 文件，由于 NetCDF 文件可能占用的空间过大，所以将 NetCDF 文件存储在本地，在数据库记录对应的存储地址。GET 请求的结构根据前端指定的文件 id，从数据表中查询对应文件的存储路劲，然后读取文件信息并处理返回给前端，数据库的表设计如所示。本系统设置 2 个表，如表 3-5 和表 3-6 所示。表 3-5 表示前端上传的 NetCDF 文件数据表，用于存储前端上传的数据文件。表 3-6 表示系统内置的数据文件表，无需前端上传，前端即可利用这些表内置的文件进行可视化，主要用于可视化效果的展示。数据库使用 mysql2 进行连接，结合 Sequelize，定义两张表存文件路劲及相关元数据。

表 3-4 后端接口描述

接口路劲	请求方式	请求体	返回数据
/api/NcFileHandler	POST	file: 上传的标量场、矢量场 NetCDF 文件	文件 id
/api/getGridData	GET	fileId: 文件 id params: 查询参数	nc 文件读取并处理后的信息

表 3-5 上传的 NetCDF 文件数据表

字段名	类型	说明
id	INT	唯一标识符，主键
file_path	VARCHAR(255)	文件存储路径
file_name	VARCHAR(255)	文件名
upload_time	DATETIME	上传时间

表 3-6 系统内置 NetCDF 文件数据表

字段名	类型	说明
id	INT	唯一标识符，主键
file_path	VARCHAR(255)	文件存储路径
file_name	VARCHAR(255)	文件名
create_time	DATETIME	添加时间

3.3 海洋环境要素场数据处理

3.3.1 NetCDF 格式数据

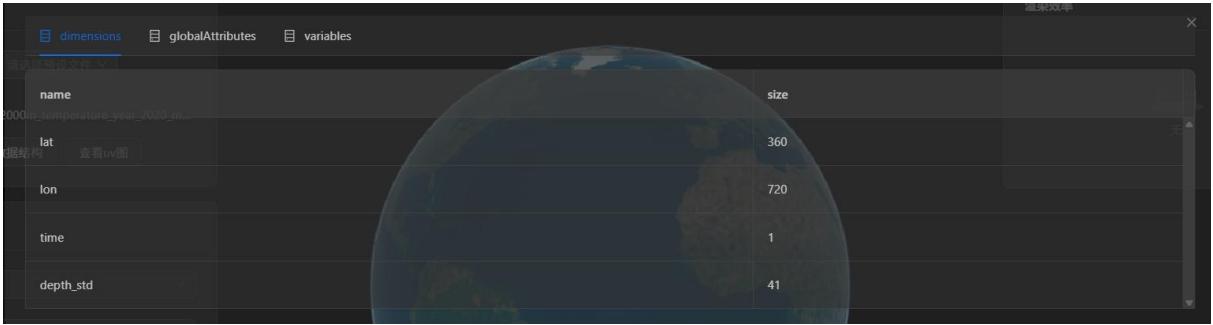
本系统基于 NetCDF 格式数据进行可视化，它由美国大气研究协会的 Unidata 项目科学家针对科学数据的特点开发的，是一种面向数组型并使用于网络共享的数据描述和编码标准。类似于 zip、jpeg、bmp 等文件格式，NetCDF 也是一种文件标准，NetCDF 主要用于存储气象科学的数据。利用 NetCDF，可以高效地存储、管理、访问和分发网络中的海量数组型数据。NetCDF 存储的数据实际上是多变量的单值函数，在 GIS 领域，常常将经度、纬度、高度、时间等作为自变量，温度、盐度、溶解氧等作为函数值。

NetCDF 文件后缀为.nc，文件中的数据结构包含维（Dimensions）、变量（Variables）和属性（Attributes）三种描述类型，其结构如下：

```
{
  NetCDF name{
    Dimensions:... //定义维数
    Variables:... //定义变量
    Attributes:... //属性
    Data:...//数据
  }
}
```

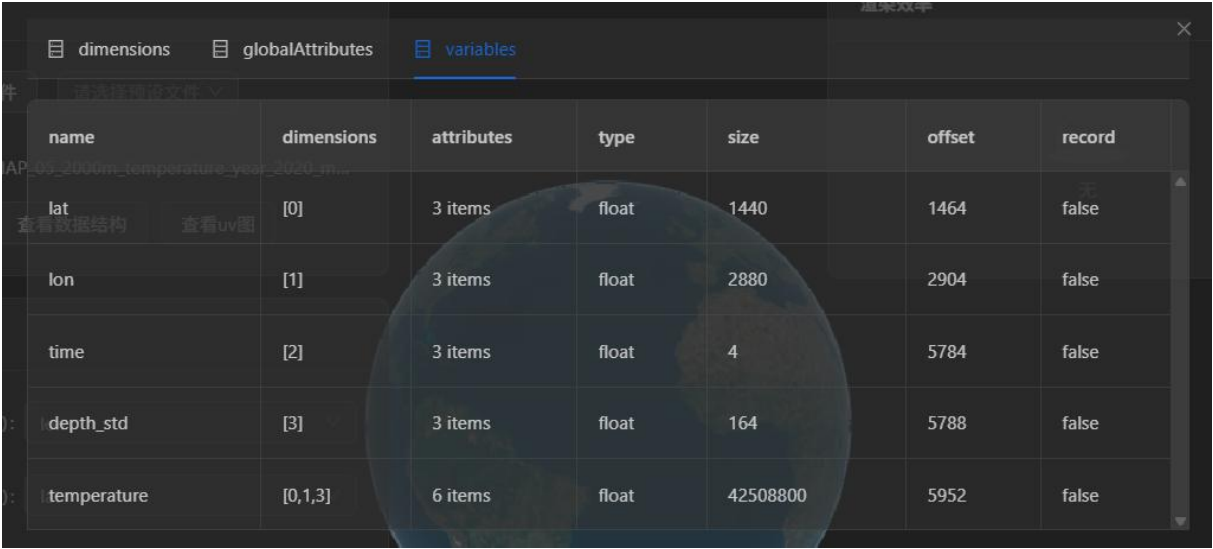
本文数据来自于中国科学院海洋研究所海洋科学大数据中心和美国国家海洋和大气管理局（National Oceanic and Atmospheric Administration, NOAA）国家气候信息中心（National Centers for Environmental Information, NCEI），均为 NetCDF 格式，主要数据类型为海面温度场数据、海面高度场数据及全球风场数据。

在 GIS 可视化阶段，比较需要关心的是维和变量信息。以温度场数据为例。其维数数据解析至系统页面如图 3-8 所示，数据表明该文件的 lon（经度）有 360 个维度，lat（纬度）有 720 个维度，depth_std 有 41 个维度，时间拥有一个维度。本系统就是基于这个信息的经纬度维度来生成纹理图片的宽高。其变量信息如图 3-9 所示，其中 lon 的索引是 0，lat 的索引是 1，depth_std 的索引是 3，温度是索引是[0,1,3]，表明这个温度产品数据是一个三层嵌套循环获取的，最外层循环是 lon（对应索引 0），下一层循环是 lat（对应索引 1），最内层循环是 depth_std（对应索引 3）。



name	size
lat	360
lon	720
time	1
depth_std	41

图 3-8 NetCDF 文件的维度信息



name	dimensions	attributes	type	size	offset	record
lat	[0]	3 items	float	1440	1464	false
lon	[1]	3 items	float	2880	2904	false
time	[2]	3 items	float	4	5784	false
depth_std	[3]	3 items	float	164	5788	false
temperature	[0,1,3]	6 items	float	42508800	5952	false

图 3-9 NetCDF 文件的变量信息

3.3.2 数据服务化

本系统基于 NetCDF.js 开源库读取 NetCDF 文件数据，此插件支持简单几行代码就获取到文件的维度、变量、属性以及数据信息，该开源库目前仅支持 NetCDF v3 版本以及下的文件，所以本系统目前也仅支持渲染 NetCDF v3 版本及以下的文件。然而读取这样获取的数据并不符合前端数据的渲染要求，NetCDF 里的变量数据均是由一维数组来表示，变量之间的关系通过索引来连接，在前端页面数据渲染中，希望获取的是一个二维数组，即外层数组里的每一项是一个一维数组，一维数组中记录了[经度，纬度，温度/盐度]这样的信息，前端获取到外层数组，就可以遍历外层数组，获取每个点的温度/盐度值，再根据这个点的位置和强度实例化几何图形。

对于有些 NetCDF 文件数据，其分辨率过高，使用离散数据实例化渲染法和点图标法渲染到本系统上会导致页面卡顿，用其他方法不会，因为离散数据实例化渲染法和点图标法需要在每个点生成几何图形，会生成大量的顶点和三角面片，其他方法不会生成

实例化图形，只是利用数据生成纹理。为了在大数据量情况下保持页面的渲染流畅，所以需要在选择离散数据实例化渲染法和点图标法时，数据处理阶段要加上采样的步骤。本系统设置的采样阈值是 5w，不超过 5w 个点时，不对数据进行采样，超过 5w 个点时候根据超出的强度进行数据的等间隔采样压缩。

数据处理的生成二维数组阶段，需要根据变量的索引值，计算经度变量和维度变量在其强度变量（温度、盐度、U、V 等）的步长，根据步长书写二维循环遍历就可以生成目标数据。生成前端渲染所需的二维数据之后，还需要剔除掉无效的数据，只需要变量二维数组，当其强度变量的大小为 0 时，将其从原数组中删除，从而实现剔除陆地的数据。

除了计算点的位置和强度信息外，还应将经纬度间隔，数据长度，经纬度原始维度等数据计算好发送给前端，其核心处理函数代码如下：

```
/**
 * 将一维或多维数据转换为二维数组，同时计算相关的采样信息。
 * @param {string} lonName - 经度变量名称。 "lon"
 * @param {string} latName - 纬度变量名称。 "lat"
 * @param {Array} u - u 分量数据数组。 [1, 2, 3, ...]
 * @param {Array} v - v 分量数据数组。 [4, 5, 6, ...]
 * @param {Array} index - 数据维度索引数组。 例如 [3, 2, 1, 4] 表示 u 和 v 分量的维度顺序。
 * @param {Object} indexMapping - 维度索引与变量数据的映射关系。 ['0': {name: 'lon', data: [1,2,3]}, '1': {name: 'lon', data: [1,2,3]},...]
 * @param {boolean} isSample - 是否进行数据采样。 true 或 false。
 * @returns {Object} 包含二维数组结果和采样信息的对象。
 */
function convertTo2DArray(lonName, latName, u, v, index, indexMapping, isSample) {
  console.log(indexMapping)
  const xObject = indexMapping[index[0]];
  xObject.demension = '一维';
  const yObject = indexMapping[index[1]];
  yObject.demension = '二维';
  const zObject = indexMapping[index[2]];
  zObject.demension = '三维';
  let wObject;
  if(Object.keys(indexMapping).length === 4) {
    wObject = indexMapping[index[3]];
    wObject.demension = '四维';
  }
  const strids = {
    '一维': yObject.data.length * zObject.data.length * (wObject?.data.length || 1),
```

```

    '二维': zObject.data.length * (wObject?.data.length || 1),
    '三维': wObject?.data.length || 1,
    '四维': 1
  };
  const lonObject = [xObject, yObject, zObject, wObject].find(item => item.name === lonName);
  const latObject = [xObject, yObject, zObject, wObject].find(item => item.name === latName);
  const originLength = lonObject.data.length * latObject.data.length;
  const sampleRate = isSample ? Math.max(1, Math.floor(originLength / 50000)) : 1; // 采样率
  const latRange = [Math.min(...latObject.data), Math.max(...latObject.data)];
  const textureWidth = lonObject.data.length / sampleRate;
  const textureHeight = latObject.data.length / sampleRate;
  let resultArr = [];
  for (let i = 0; i < lonObject.data.length; i += sampleRate) {
    for (let j = 0; j < latObject.data.length; j += sampleRate) {
      const lon = lonObject.data[i];
      const lat = latObject.data[j];
      const fIndex = i * strids[lonObject.demension] + j * strids[latObject.demension];
      resultArr.push([lon, lat, u[fIndex], v[fIndex]]);
    }
  }
  const sampledLons = [...new Set(resultArr.map(item => item[0]))].sort((a, b) => a - b);
  const sampledLats = [...new Set(resultArr.map(item => item[1]))].sort((a, b) => a - b);
  const lonDistance = sampledLons.length > 1 ? sampledLons[1] - sampledLons[0] : 0;
  const latDistance = sampledLats.length > 1 ? sampledLats[1] - sampledLats[0] : 0;
  resultArr = resultArr.filter(item => item[2] && item[3]);
  return {resultArr, dataInfo: {lonDistance, latDistance, sampleRate, originLength, textureWidth, textureHeight, latRange}};
}

```

3.4 海洋环境要素标量场可视化实现

3.4.1 离散数据实例化颜色映射法

(1) 点渲染

点渲染即利用点作为实例，表示其位置，点的颜色表示位置的强度。本系统基于 Cesium 的 Entity 类实现点的位置和颜色渲染，Entity 类允许设置点的颜色，像素大小进行参数化建模。颜色采用 HSL（Hue Saturation Lightness，色相-饱和度-亮度）模型来计算，色相固定为红色色相，饱和度根据该点的强度由小到大赋予[0,100%]（线性映射），亮度根据该点的强度由大到小赋予[0.5,1]（线性映射）。以海面温度场数据为例，渲染效果如图 3-10 所示。

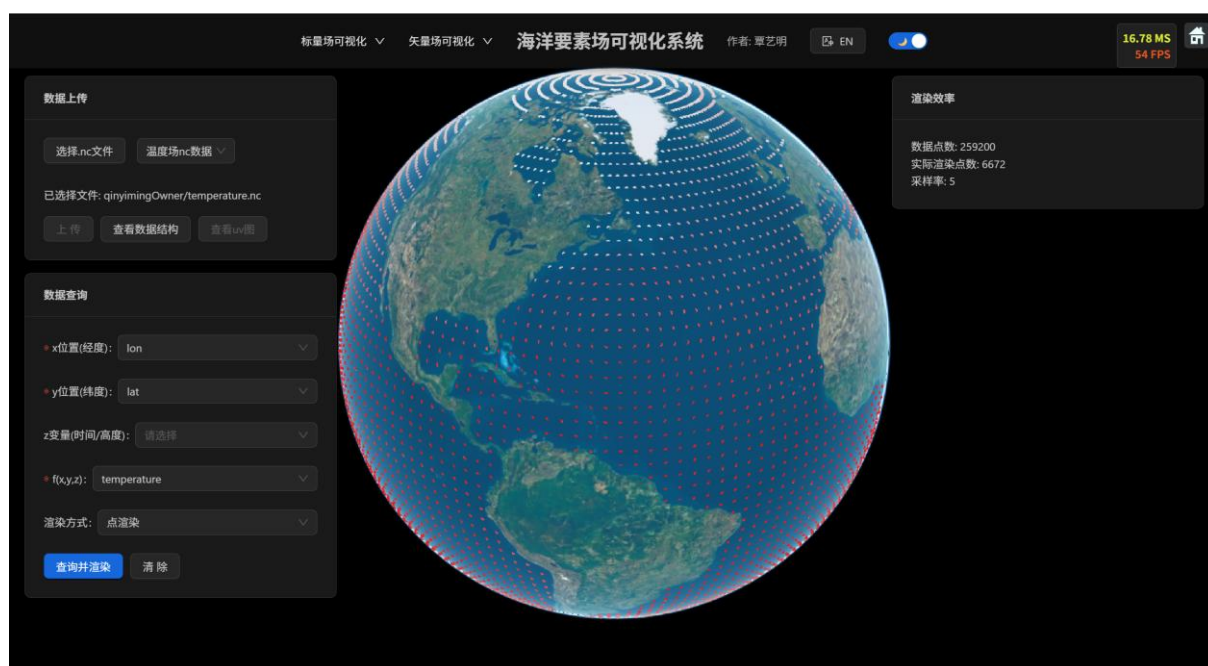


图 3-10 点渲染结果图

(2) 柱渲染

柱渲染即利用作为实例,表示其位置,点的颜色表示位置的强度。本系统基于 Cesium 的 `ConstantProperty` 类实现柱的位置和颜色渲染,颜色采用 HSL 模型来计算,色相固定为红色色相,饱和度根据该点的强度由小到大赋予 $[0,100\%]$ (线性映射),亮度根据该点的强度由大到小赋予 $[0.5,1]$ (线性映射)。以海面温度场数据为例,渲染效果如图 3-11 所示。

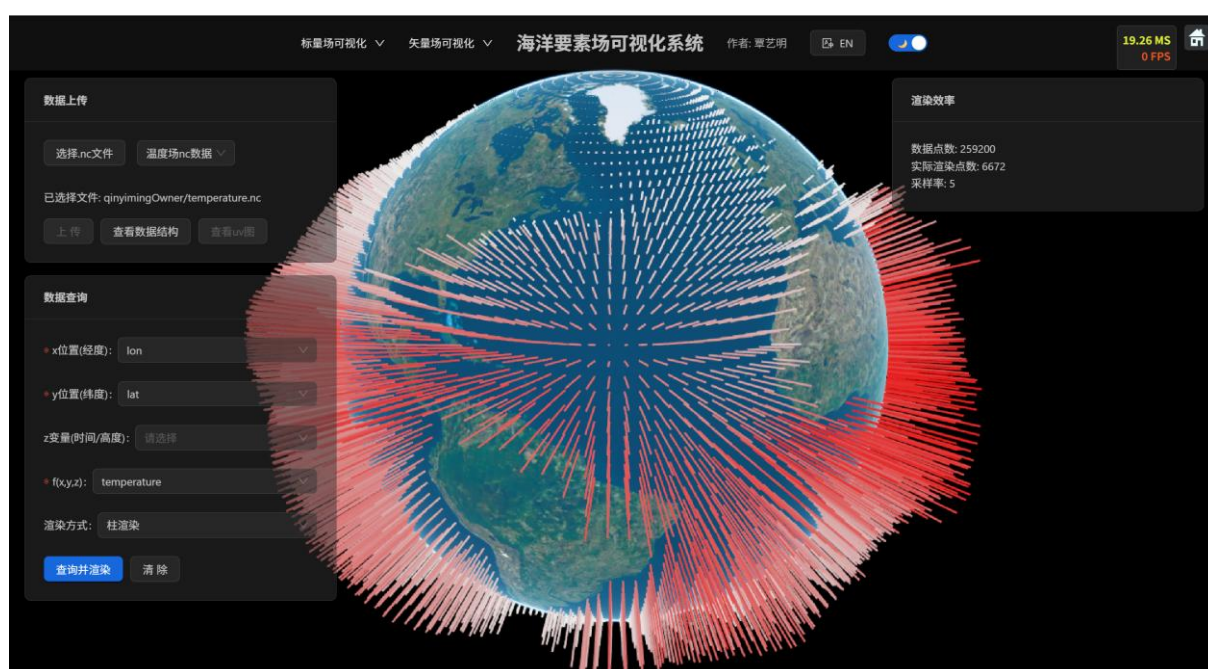


图 3-11 柱渲染效果图

(3) 多面体渲染

多面体渲染即利用多面体为实例，表示其位置，多面体的颜色表示位置的强度。本系统基于 Cesium 的 GeometryInstance 类实现几何的位置和颜色渲染。

本系统基于 BoxGeometry 类，实现几何体实例的生成，使用 Cesium 提供的 ENU（东北天）变换矩阵将几何体实例偏移 to 地球表面并使其朝向天空，采用 HSL 模型来计算几何体实例的颜色，色相根据该点的强度由小到大赋予[0,1]（红到紫，线性映射），饱和度固定为 1，亮度固定为 0.5，将颜色作为几何体实例的颜色数据传入到几何体的顶点着色器。设置材质外观应用到几何体实例上，生成渲染图元，在片元着色器上，应用前面设置的顶点颜色。以海面温度场为例，渲染效果如图 3-12 所示。

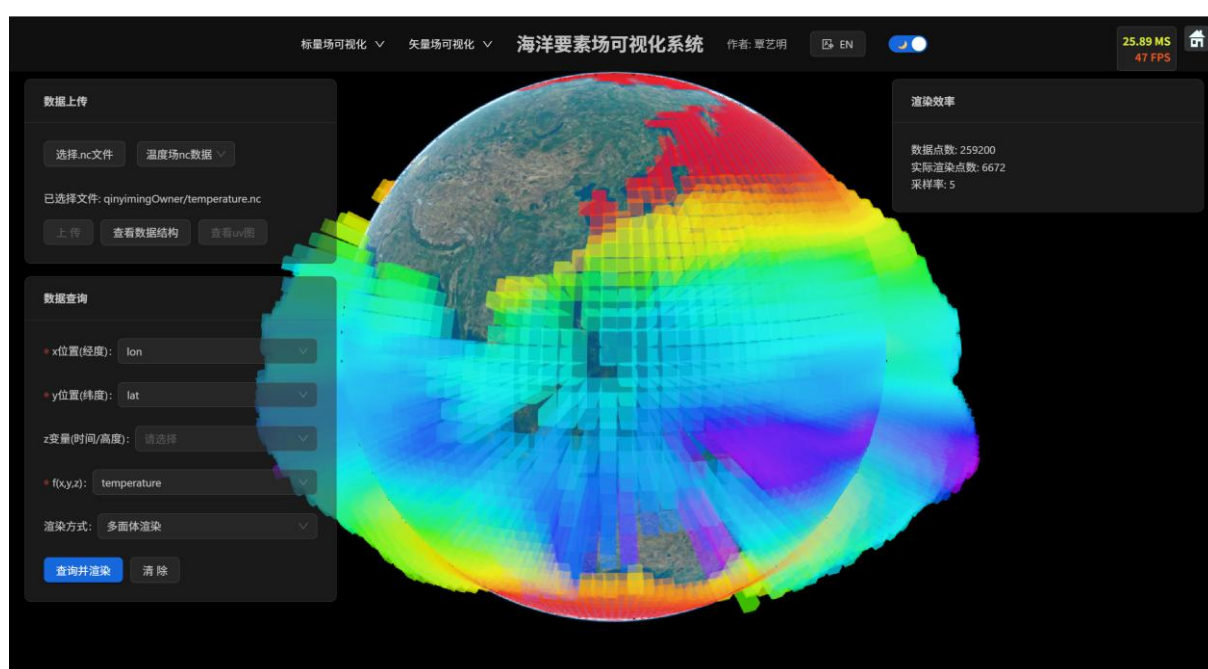


图 3-12 多面体渲染结果图

3.4.2 纹理映射法

以海面温度场数据为例，纹理渲染法的具体步骤为：

步骤 1：生成温度场纹理图。根据 NetCDF 格点数据生成一张纹理图，纹理图 R 通道存储强度信息。设置生成纹理图片时，本系统使用的是 HTML5 的 canvas 元素绘制的，canvas 元素绘制的精度有限，仅支持 Uint8 类型的数据，故只能存储 0 到 255 范围内的数据，所以需要将强度数据（温度）先归一化再映射到 0 到 255 范围内，依据温度网格数据，将强度信息保存在图片 R 通道上，生成的纹理图如图 3-13 所示。



图 3-13 温度场数据颜色纹理图

步骤 2: 生成颜色纹理图。本系统中采用的颜色级别映射表如表 3-7 所示。分为 13 个级别, 生成渐变颜色纹理图时, 首先需要将温度归一化, 计算每个级别对应温度数据, 采用渐变采样/分段采样的方法, 即可生成纹理图, 生成的分段纹理图如图 3-14 所示, 渐变纹理图如图 3-15 所示。

表 3-7 RGB 颜色映射等级表

级别	RGB	颜色
1	(115,70,105)	
2	(202,172,195)	
3	(162,70,145)	
4	(143,89,169)	
5	(157,219,217)	
6	(106,191,181)	
7	(100,166,189)	
8	(93,133,198)	
9	(68,125,99)	
10	(128,147,24)	
11	(243,183,4)	
12	(232,83,25)	
13	(71,14,0)	

图 3-14 分段纹理图



图 3-15 渐变纹理图

步骤 3: 生成几何体实例。使用 Cesium 提供的 `RectangleGeometry` 类生成一个覆盖全球的几何图形实例。

步骤 4: 生成几何体外观。使用 Cesium 提供的 `AppearanceMatial` 类, 应用到步骤 3 生成的几何图形实例, 添加到场景当中。关于 `AppearanceMatial` 类内部默认封装的着色器代码, 前文理论部分已经描述, 可以看出 Cesium 默认封装的着色器代码, 顶点着色器符合纹理渲染法的需求, 片元着色器是不符合的, 所以需要修改其片元着色器代码。将温度场纹理图像和颜色纹理图作为全局纹理变量传入到着色器渲染管线当中, 在片元着色器部分接收使用。

步骤 5: 片元着色器内插值运算获取温度值。GPU 中片元着色器对图片纹理进行采样过程中, 以每个像素点为单位, 纹理图片精度是不足的, 所以需要采样获取该点的温度值。可使用最邻近/双线性/双三次插值的方法获取当前片元的强度信息, 根据强度从颜色图例纹理中获取颜色, 将颜色取样结果从片元着色器输出。

基于以上步骤, 本系统在使用双线性插值方法下, 完成渐变颜色温度场渲染(如图 3-16 所示)和分段颜色温度场的渲染(如图 3-17 所示)。

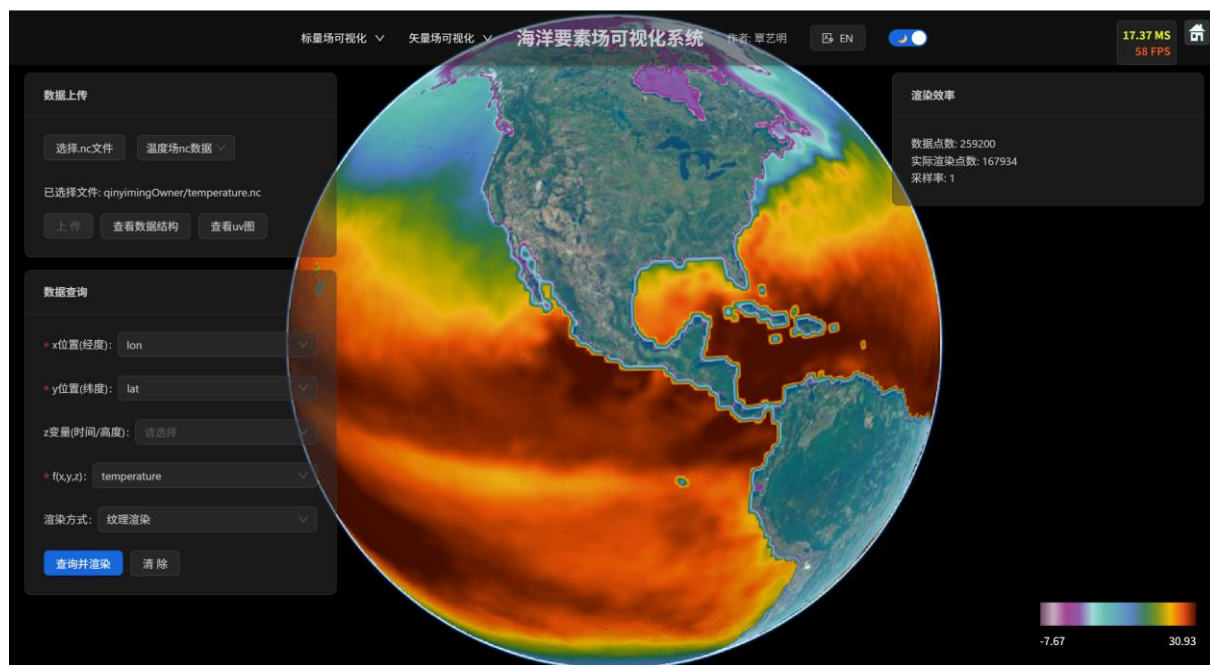


图 3-16 渐变颜色温度场渲染结果

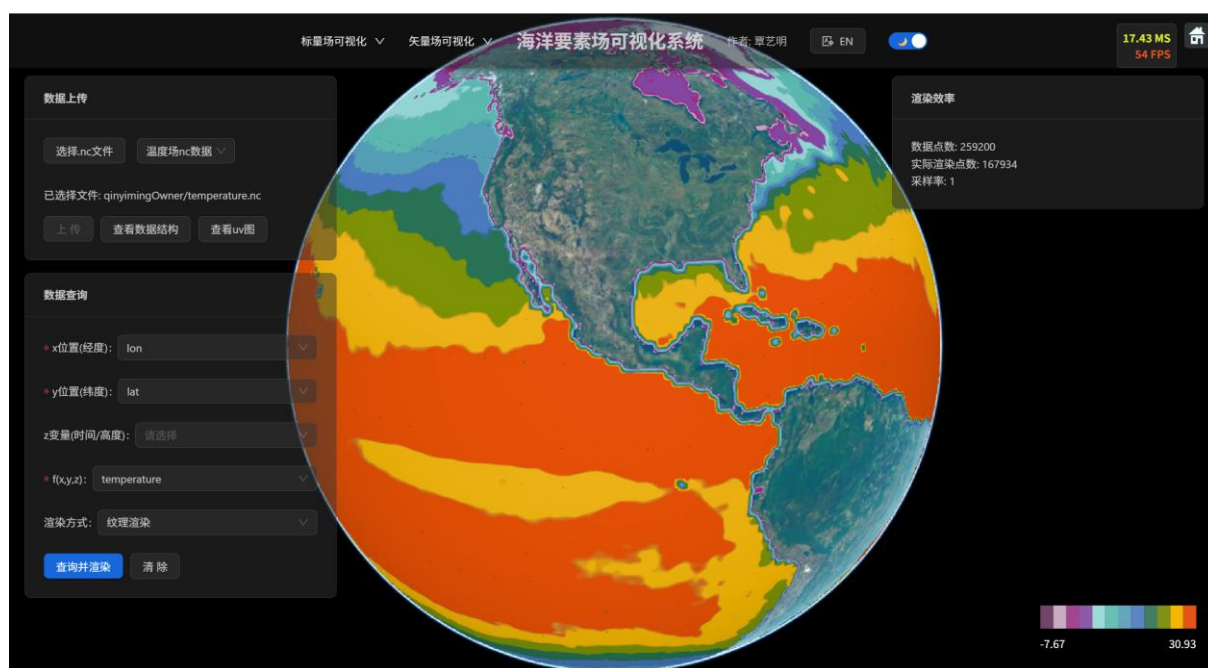


图 3-17 分段颜色温度场渲染结果

通过对比分析,我认为渐变颜色的可视化更好,更改体现更多的温度信息以及变化信息,分段更像是一种聚类结果,使用分段颜色一定程度上降低了插值算法的效果,所以本系统默认采用渐变颜色渲染的方法

颜色纹理采用渐变颜色条,分别采用最邻近、双线性、双三次插值的方法插值温度纹理数据,其结果分别如图 3-18、图 3-19、图 3-20 所示。

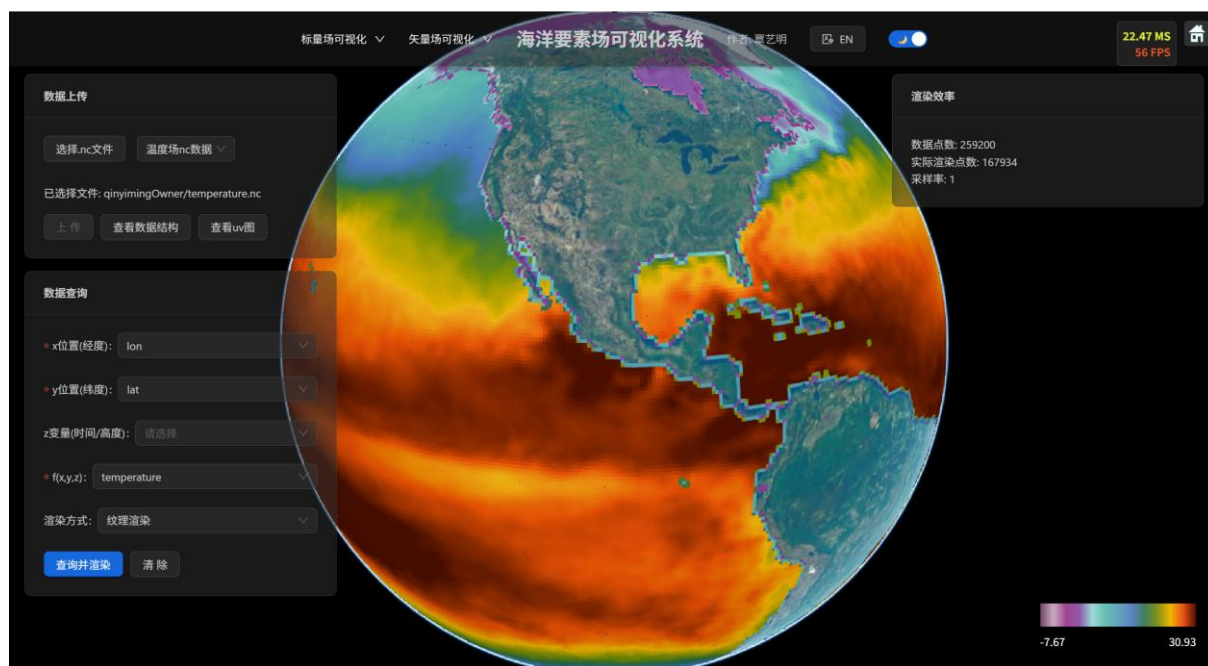


图 3-18 纹理最临近插值法渲染结果

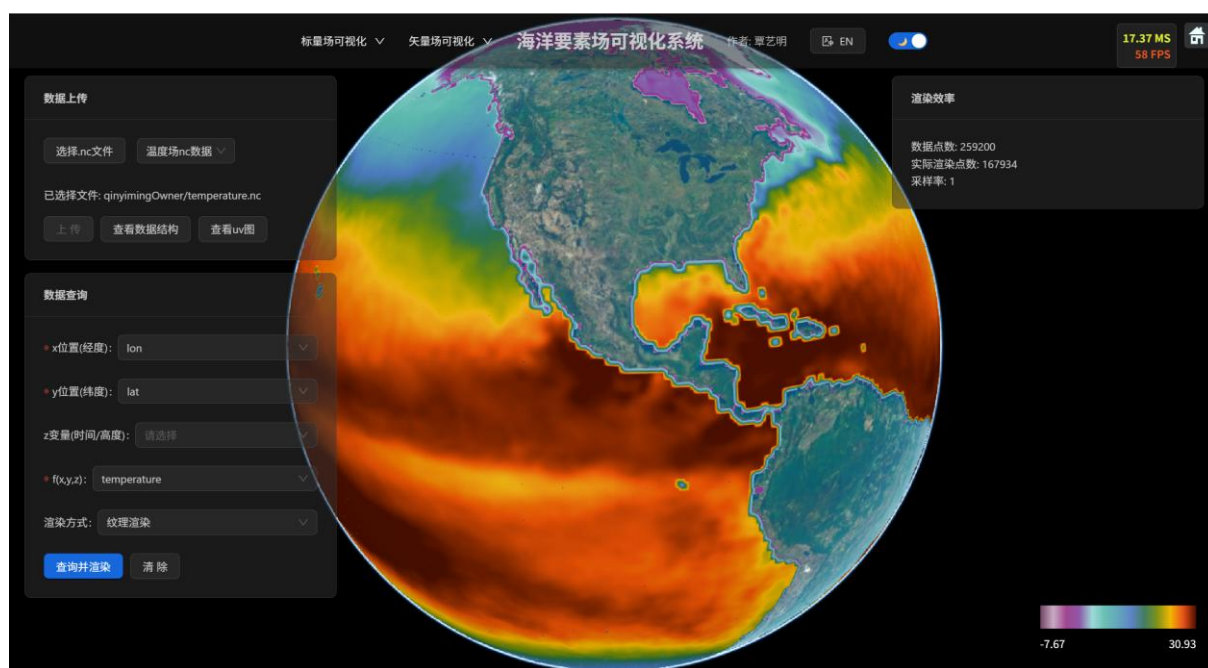


图 3-19 纹理双线性插值渲染结果图

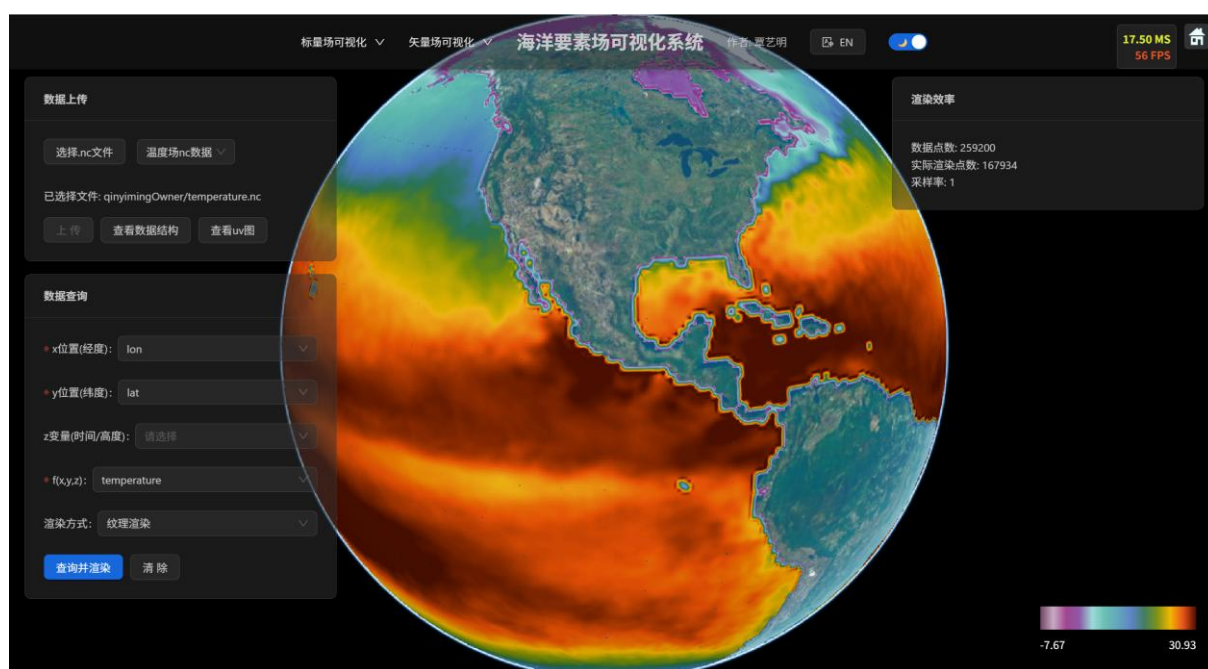


图 3-20 纹理双三次插值渲染结果图

通过三种插值渲染结果比对，最近邻的渲染效果有明显的网格化，变形比较突兀，总体看起来比较模糊，双线性插值和双三次插值渲染结果差别不大，两者在渲染效果上没有明显区别，后者的插值结果更精确，但本系统未开发点击图形获取纹理值的功能，所以使用双三次插值的意义不大，反而增加了 GPU 的运行负担。

经过综合取舍，本系统最终采用双线性插值、颜色渐变的方法作为系统的纹理映射法，其渲染效果如图 3-19 所示。

3.4.3 海面波浪生成法

以海面高度场数据为例，纹理渲染法的具体步骤为：

步骤 1：生成高度场纹理图。根据 NetCDF 格点数据生成一张纹理图，纹理图 R 通道存储海平面的高度信息。生成方法与前文中的温度场纹理图生成的方法一样，生成的结果如图 3-21 所示。



图 3-21 高度场数据颜色纹理图

步骤 2：生成几何体实例。与前文中纹理映射法的步骤 3 一致。

步骤 3：生成几何体外观。与前文中纹理映射法的步骤 4 差不多，区别在于此处不将颜色纹理图传入到着色器渲染管线。

步骤 4：偏移几何体实例顶点。在顶点着色器内，获取高度场纹理图，根据双线性插值算法获取该点的实际海面高度值，根据这个值使几何体实例的顶点在法线方向上进行偏移。

步骤 5：根据 UV 生成视线向量。在片元着色器内，根据 UV 坐标生成眼睛视线向量，使用旋转矩阵保证视线射向海洋而不是天空。

步骤 6：根据二分法计算视线与海平面的交点。二分法迭代步进的过程中，实时构建海浪高度场，设置二分法的迭代次数模拟光线前进花费的时间。将二分迭代结果作为视线与海平面的交点。

步骤 7：计算海面的法线。根据海面的高度计算该片元着色器的法线，高度越高，其法线越垂直于天空。

步骤 8: 计算海面反射颜色。计算天空颜色, 根据视线方向生成渐变色天空, 低角度偏蓝, 高角度偏白。利用海水颜色生成模型, 计算使用光线追踪法算法计算海面的反射颜色, 将颜色从片元着色器输出。

依据上述步骤, 设置二分迭代次数为 8 次; 海水基底色为 RGB (12,38,63); 水体反光色设置为 RGB (102,153,229); 高光衰减指数设置为 60; 高度影响系数设置为 0.18。其渲染结果如图 3-22 所示。

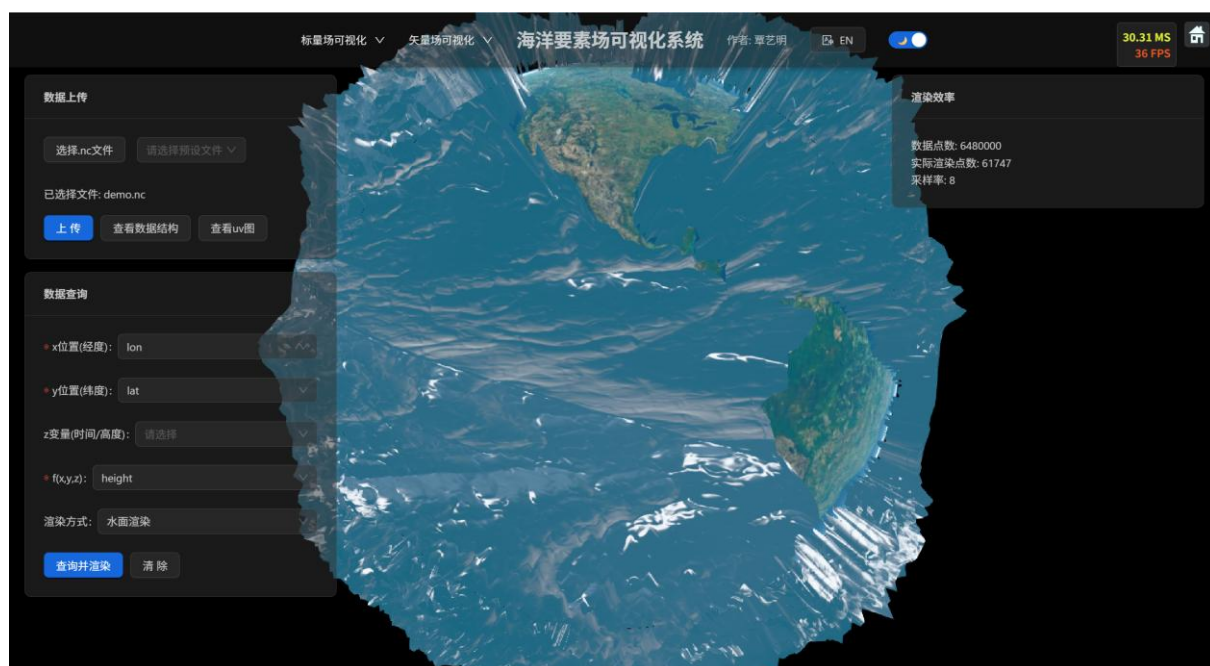


图 3-22 海面波浪生成法渲染结果

3.5 海洋环境要素矢量可视化实现

3.5.1 点图标法

以全球风场数据为例, 点图标渲染法的具体步骤为:

步骤 1: 生成几何体实例。此处的实例生成与前文中的纹理映射法和海面波浪生成法不同, 此处需要遍历网格数据生成多个矩形几何体实例 (使用 Cesium 的 RectangleGeometry 类), 根据网格数据的分辨率, 设置几何体的宽高参数, 使用 ENU 变换矩阵将每个几何体实例变换到地球表面使其朝向天空。

步骤 2: 旋转 UV 纹理坐标。给矩形几何体的 UV 纹理坐标应用旋转变换矩阵, 旋转的角度根据网格数据的 UV 决定, 计算 UV 的模, 归一化后作为强度绑定到矩形几何体的颜色 R 通道分量上。

步骤 3: 设置纹理图片传入着色器渲染管线。使用一个箭头图片作为纹理, 使用渐

变方法生成颜色纹理（与纹理映射法的步骤 2 一致），将两个纹理与矩形几何体一同进入着色器渲染管线。

步骤 4：片元着色器应用传入的纹理。在片元着色器阶段，将箭头纹理应用到矩形几何体上，从几何体的颜色 R 通道获取风速强度，根据风速的强度到颜色纹理采样颜色，将采样颜色和箭头纹理混合，从片元着色器输出。

依据上述步骤，其渲染结果如图 3-23 所示。

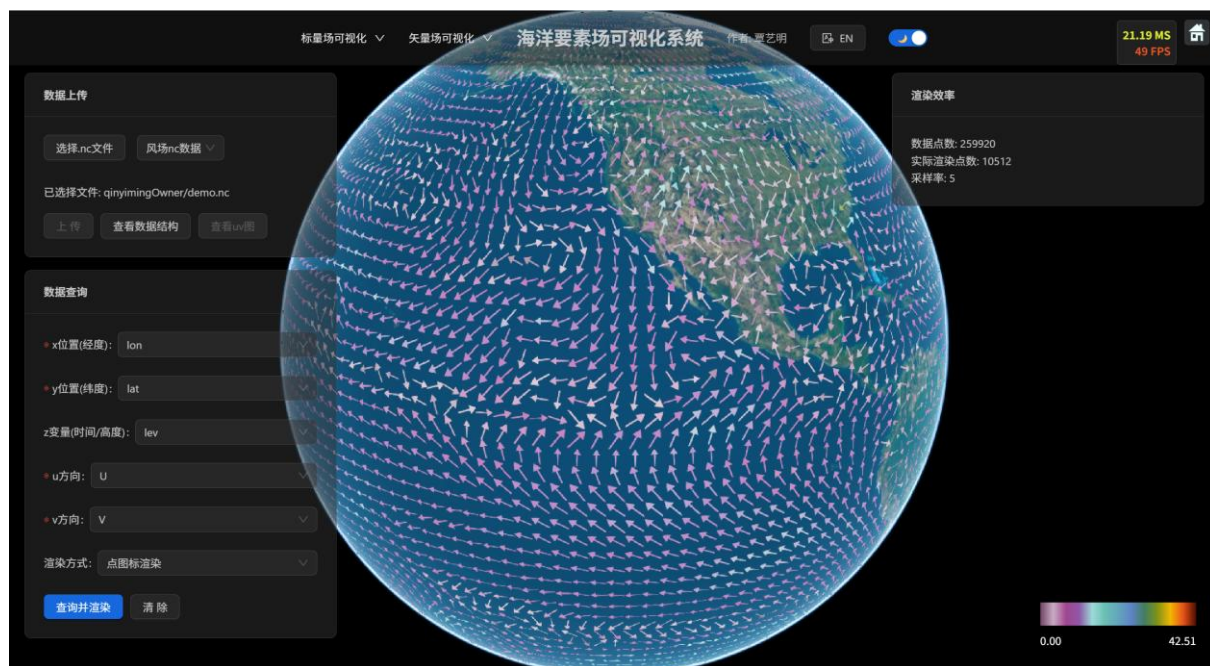


图 3-23 点图标法渲染结果

3.5.2 粒子系统法

以全球风场数据为例，粒子系统渲染法的具体步骤为：

步骤 1：搭建渲染流水线。自定义一个 Primitive 类（Cesium 提供的类），在其原型链上定义 update 方法，在 update 方法里生成 DrawCommand（Cesium 提供的类）指令推入到 Cesium 的帧状态管理对象的命令列表中，实例化 4 个自定义 Primitive 对象，分别掌管粒子速度、粒子位置、粒子后处理、粒子渲染，搭建渲染流水线，在 Cesium 场景中添加这 4 个实例。

步骤 2：生成 UV 方向纹理。根据网格数据，生成 U 方向的纹理（如图 3-24 所示）和 V 方向（如图 3-25 所示）的纹理，传入到粒子速度计算管线；粒子速度计算管线内部使用二阶龙格库塔方法计算粒子速度，将片元着色器输出到一个全局变量当中，供给其他绘制指令使用。

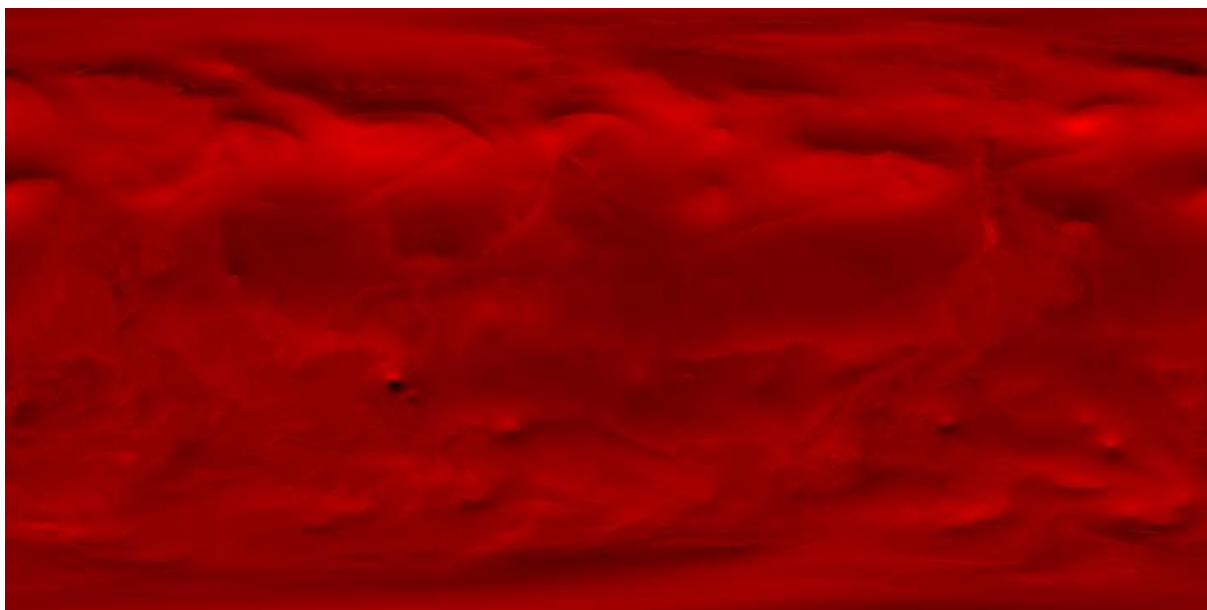


图 3-24 U 方向编码纹理图

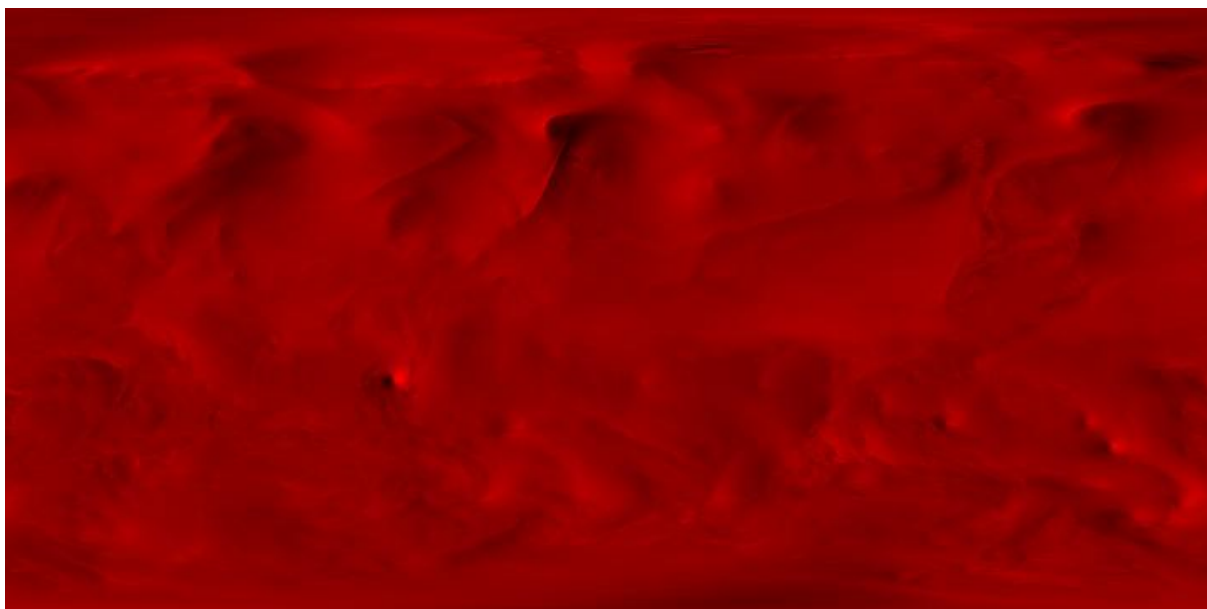


图 3-25 V 方向编码纹理图

步骤 3: 生成粒子并设置生命周期。在全球范围内生成 $1w$ 个粒子, 随机生成粒子的初始化位置纹理如图 3-26 所示, 经度信息编码至 R 通道, 纬度信息编码至 G 通道。根据粒子速度更新粒子的位置网格, 引入粒子掉落概率, 根据速度和随机性设置粒子的消亡, 速度越大消亡的几率越大, 当粒子超出地球球体边界时, 也需要设置粒子消亡, 消亡即在新的随机位置生成粒子, 以保证粒子的总数维持在固定的数目。

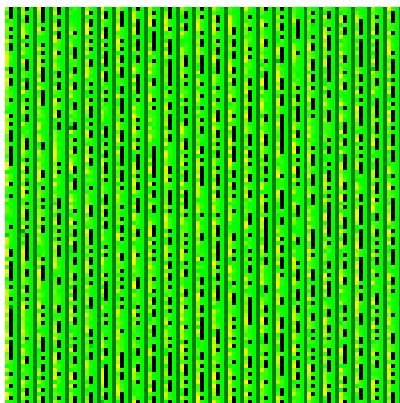


图 3-26 粒子初始位置编码纹理图

步骤 4: 实例化粒子几何图形, 根据粒子的位置网格渲染到屏幕上。

步骤结束之后循环重复执行步骤 3 和步骤 4。其渲染结果如图 3-27 所示。

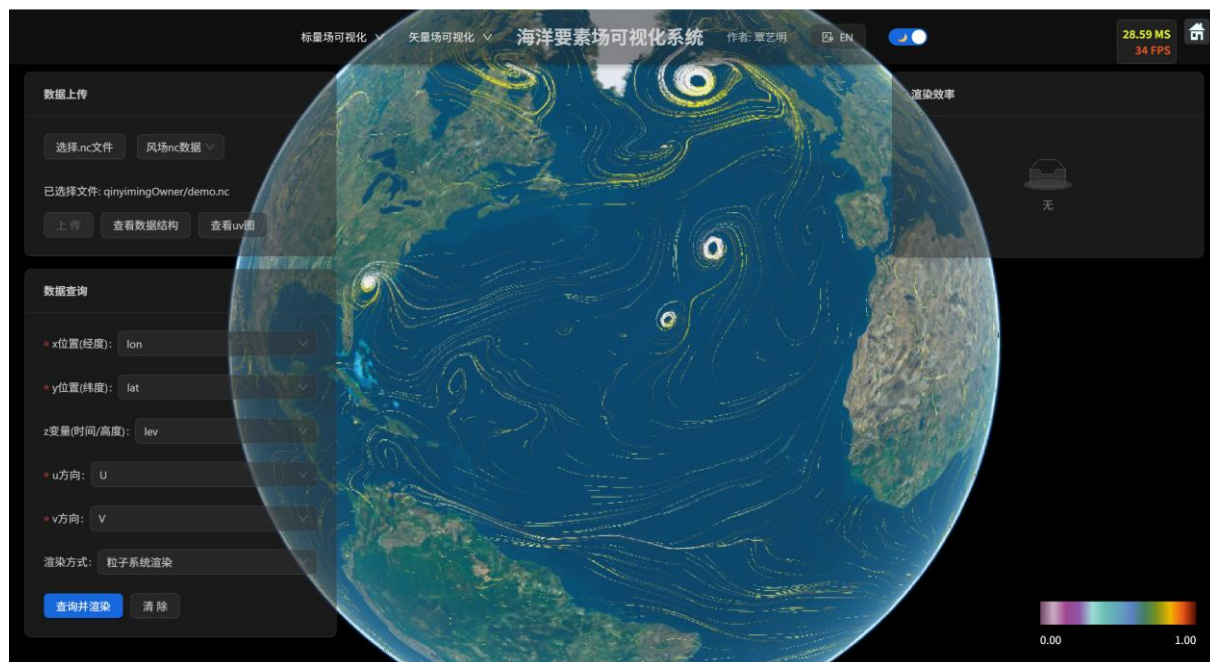


图 3-27 粒子系统法渲染结果

第4章 总结与展望

4.1 总结

随着海洋数据应用需求的不断提升，基于 Web 的三维可视化技术日益成为海洋数据分析与展示的重要手段。本文以 NetCDF 格式的海洋标量场、矢量场数据为研究对象，设计了海洋环境要素场可视化方法，设计并实现了面向海洋信息的三维可视化平台，验证了设计的可视化方法。通过结合国内外海洋模型与数据可视化的发展现状，成功实现了高效、交互式的海洋数据可视化应用，为海洋环境监测与管理提供技术支撑。

本文主要工作与结论如下：

（1）深入分析了国内外海洋数值模型及海洋标量、矢量数据的可视化技术现状，全面梳理了常用的可视化方法，为本项目的技术选型提供了坚实基础。

（2）针对海洋标量场的可视化，本研究总结了离散数据实例化颜色法、纹理映射法、海面波浪生成法，强调了不同方法在实现中的关键环节。前两种方法主要基于颜色映射模型，可用于表现海洋环境中的温度、盐度、叶绿素浓度等标量特征，而海面波浪生成方法则专注于海面动态纹理的真实还原。

（3）针对海洋矢量场的可视化，本研究总结了点图标法与粒子系统法两种常用策略。点图标法适用于静态场景的矢量方向及大小展示，粒子系统法则通过动态粒子流形象表现海流等动态矢量信息。这两种方法的结合极大丰富了海洋矢量数据的表现形式。

（4）针对 NetCDF 格式数据的组织结构，本文详细介绍了其存储方式与数据处理流程。特别说明了如何将原始 NetCDF 数据转换成系统支持的格式，包括数据的索引、提取与预处理，为后续数据载入提供了技术方案。

（5）在系统实现方面，本文设计并开发了一个高效的 Web 端海洋数据三维可视化系统。系统采用 B/S 架构设计，在实验环境下，支持海洋标量和矢量场的海量数据（支持 5 万级别）实时渲染而不卡顿。界面设计兼顾 PC 端和手机端，支持多终端操作；同时实现了中英文切换、主题切换等人性化功能，提升用户体验。通过集成 Cesium、纹理渲染等技术，实现海洋数据的立体沉浸式展示，支持多视角浏览、动态效果、交互查询等功能，满足不同应用场景的需求。

4.2 展望

由于开发者技术水平有限，本文所述海洋环境要素场可视化系统还有以下不足之处，有待于进一步研究。

(1) 优化后端数据处理。本系统目前仅支持 NetCDF v3 及以下版本数据的可视化，未来应当实现多版本的 NetCDF 格式数据的可视化处理，并且支持其他格式数据如 GRIB、geoJson、tiff 等。

(2) 优化渲染性能。本研究数据层面的复杂计算放置在后端，客户端中复杂的渲染主要在着色器渲染管线也就是 GPU 中计算，已最大程度优化可视化性能，然而仍具备优化空间，未来可使用 WebGPU 进行可视化，本系统目前是基于 WebGL 进行数据可视化的，而 WebGPU 的渲染效率是 WebGL 的三倍，未来随着 WebGPU 适配到各 Web 端，WebGPU 渲染海洋环境要素场或将成为主流，目前 Babylon.js 渲染引擎已支持 WebGPU，本研究目前不使用 Babylon.js 而使用 Cesium 是考虑到了目前大部分 Web 浏览器还未适配 WebGPU，并且 Cesium 对地理数据的可视化有更好的支持。

(3) 丰富系统可视化方法。目前本系统支持的可视化标量可视化方法细分来看有 5 中，矢量可视化有 2 中，颜色映射仅支持线性渐变映射，未来应增加可视化方法，并且同时支持选择各种颜色条进行可视化。

致 谢

时光飞逝，四年的大学生活如白驹过隙，回首那些点点滴滴，心中满是感慨与感激。在此，我衷心感谢所有陪伴我成长、给予我帮助的人们，是你们让我的学习和生活变得丰富多彩、温暖无比。

首先，我要深深感谢我的家人——父母亲人。感谢你们无条件的支持与养育之恩，你们的恩情我无以为报。你们的爱是我不断前行的动力，我会继续努力，争取早日用实际行动回报你们的期望。

其次，感谢我的老师们。感恩李连伟老师的悉心指导，从论文选题到毕业设计的每一个环节，李老师都一丝不苟、热心帮助，在我遇到困难时给予我宝贵的建议和鼓励。还要感谢薛存金老师，为我提供了宝贵的实习机会。感谢所有授课老师和辅导员们，你们的教诲与关怀让我成长许多。

我还要特别感谢好未来公司、华云三维公司。让我在生产一线学习到了宝贵的开发经验。入职带教老师在我实习期间的悉心指导，让我不仅拓宽了视野，更提升了实践能力。与你们共事的每一天，我都感到非常幸运和温暖。

最后，感激所有审阅我论文的专家老师们，你们的批评指正让我更加完善自己，也为我的学术旅程增添了宝贵的指导。

参考文献

- [1] 汪小琪.基于Web的海洋环境数据可视化方法研究[D].中国石油大学(华东),2021.
- [2] Sérgio Lopes. Visualization of Scalar Data Volumes[J]. Rangeland Ecology & Management, 2014, 67(5):455-467.
- [3] 杨佳.全球气象海洋标量场数据组织与可视化关键技术研究[D].国防科技大学,2022.
- [4] 陈超.海洋标量场数据三维可视化方法研究与实现[D].国防科学技术大学,2012.
- [5] Pragensia A I. Color Mapping on Common Use-Cases in a Web Application[J]. Energy Fuels, 2015, 24(8):4206-4214.
- [6] 乔洁雯, 陈伟. 自适应颜色映射及其在空洞演化可视化中的应用[J]. 计算机应用, 2020, v.40;No.358(06):237-246.
- [7] 谢习华,王刚,辛涛,等.基于SLIC和改进区域生长的非结构化道路识别[J].计算机工程与应用,2022,58(14):210-218.
- [8] 张广宇.高分辨率海洋标量数据动态可视化方法研究[D].青岛大学,2022.
- [9] 黄国满.等值线三维显示及其并行算法[J].测绘科学,2001,(02):20-22+45-2.
- [10]刘丁.海底观测可视化信息系统的初步研究[D].同济大学,2024.
- [11]陈超,王文珂,王怀晖,等.一种海底地形与底质的三维融合可视化方法[J].系统仿真学报,2012,9 (24):1936-1939.
- [12]王想红,刘纪平,王亮,等.海洋标量数据的三维动态可视化系统研究[J].海洋科学,2013(07):92-96.
- [13]宋涛,庄雷,景晨凯.基于动态域等值线的多目标环绕跟踪方法研究[J].小型微型计算机系统,2018,8.
- [14]郑红波,陈杰,秦绪佳,等.气象多要素数据的垂直剖面可视化[J].小型微型计算机系统,2021,42(11):2350-2355.
- [15]Fan Y, Lv X Q, Zhang J K. Research and Realization of Flow Field Dynamic Visualization Based on Geometric Shader[J]. Computer engineering and Application,2019,55(9):157-161.
- [16]Wang S, Wu B, Wu T D. Survey on Perception Enhanced Flow Visualization[J]. Journal of Computer-Aided Design&Computer Graphics,2018,30(1):30-43.

- [17]郭晓非,朱俊利,万剑华,等.基于Cesium的海洋环境要素三维可视化研究[J].海洋科学,2021,45(05):130-136.
- [18]K Lu, H Shen, T Peterka. Scalable computation of stream surfaces on large scale vector fields: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, New Orleans, Louisiana, 2014[C]. IEEE Press.
- [19]J Tao, C Wang. Semi-Automatic Generation of Stream Surfaces via Sketching[J]. IEEE Transactions On Visualization and Computer Graphics, 2018,24(9):2622-2635.
- [20]侯力华.基于改进粒子系统的海流可视化技术研究[实现[D].哈尔滨工程大学,2020.
- [21]W T Reeves, R Blau. Approximate and probabilistic algorithms for shading and rendering structured particle systems[J]. Acm Siggraph Computer Graphics, 1985,19(3):313-322.
- [22]周隽文,林广发.基于粒子系统的地下综合管廊天然气泄漏模拟[J].计算机技术与发展,2024,34(06):207-213.
- [23]闫鹏,温荣臻,盛庆红,等.近地场景下爆炸型烟雾弹结构仿真方法[J].兵工学报,2024,45(09):3125-3134.
- [24]郝学娟.计算机基于粒子系统的烟花模拟[J].电子元器件与信息技术,2022,6(02):188-189+192.
- [25]刘恒星.基于粒子系统的Web数字地球海洋矢量场数据动态可视化方法[D].中国科学院大学(中国科学院遥感与数字地球研究所),2017.
- [26]Jobard B, Lefer W. Creating evenly-spaced streamlines of arbitrary density[J]. Eurographics Workshop on Visualization, 2001.
- [27]张雅静.基于三角网格的海洋标/矢量数据可视化方法研究及应用[D].中国石油大学, 2021.
- [28]张永强.基于WebGL的全球风场动态可视化方法研究[D].安徽理工大学,2022.
- [29]孙强,王世航,鹿凤.网络环境下海洋标量场数据三维可视化研究[J].现代电子技术,2019,42(08):104-108.
- [30]Cesium Team. 3D Tiles Next: Advancing 3D geospatial standards[J]. Cesium Blog, 2023.